

COS2102

บทที่ 1

หลักการเขียนโปรแกรมเชิงอ็อบเจกต์

หลักการในการสร้างซอฟต์แวร์ที่มีคุณภาพ

- การกำหนดรูปแบบและกฎเกณฑ์ (Rigor and Formality)
 - Formal language, Programming language
 - กฎเกณฑ์ที่ชัดเจน
- การแบ่งปัญหาเป็นกลุ่มย่อย (Separation of Concerns)
 - แบ่งตามเวลา (time) เช่น life cycle
 - แบ่งตามคุณภาพ (quality) เช่น efficiency, correctness
 - แบ่งตามมุมมอง (view) เช่น data flow, class diagram
- ความเป็นโมดูล
 - ความสัมพันธ์ภายในโมดูลควรมีความยึดเกาะสูง (high cohesion)
 - ความสัมพันธ์ระหว่างโมดูลควรมีความสัมพันธ์ที่ต่ำ (low coupling)

หลักการในการสร้างซอฟต์แวร์ที่มีคุณภาพ

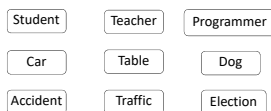
- การกำหนดสาระสำคัญ (Abstraction)
 - การแยกรายละเอียดส่วนที่สำคัญออกจากส่วนที่ไม่สำคัญ
 - เช่น ระบบบัญชี (นักบัญชี กับ โปรแกรมเมอร์ จะกำหนดสาระสำคัญไม่เหมือนกัน)
 - มีการกำหนดสาระสำคัญให้กับคลาส
 - แต่ละคลาสมีการซ่อนแอตทริบิวต์และการทำงานของแต่ละฟังก์ชันเอาไว้
 - ผู้ใช้มองเห็นแค่ส่วนของอินเทอร์เฟซที่เห็นเฉพาะส่วนที่คลาสอนุญาตให้มองเห็นเท่านั้น
- ความง่ายต่อการเปลี่ยนแปลง (Anticipation of change)
 - เช่น สามารถเปลี่ยน class ได้โดยไม่กระทบกับระบบใหญ่ทั้งหมด

หลักการในการสร้างซอฟต์แวร์ที่มีคุณภาพ

- การกำหนดกฎเกณฑ์ทั่วไป (Generality)
 - สามารถนำคลาสที่มีอยู่ส่วนมาใช้ในการพัฒนาระบบได้
 - คลาสที่นิยามขึ้นซึ่งสามารถถูกนำไปใช้โดยผู้อื่นได้ด้วย
- การเพิ่มขยายได้ (Incrementality)
 - เช่น inheritance

อ็อบเจกต์คืออะไร

- ทุกสิ่งทุกอย่างที่เราพูดถึง อ็อบเจกต์ (object)



- Object is a real-world thing or an abstraction of that thing.

อ็อบเจกต์คืออะไร

- Real-world object
 - จับต้องได้ (tangible object)
 - เช่น โต๊ะ, เก้าอี้, คน, สุนัข
 - จับต้องไม่ได้ (Intangible object)
 - นามสกุล
 - อันเดียว อาจจะมีหลายคำได้ทั้งคำซ้ำและคำซ้ำ
 - เหตุการณ์ที่เกิดขึ้น
 - การเลือกตั้ง การจดทะเบียน จูเนียร์
 - การมีปฏิสัมพันธ์ระหว่างกัน
 - สัญญาหรือข้อตกลงร่วมกัน

อ็อบเจกต์คืออะไร

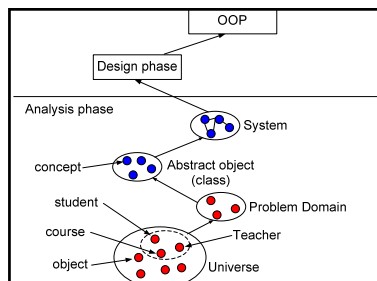
- อ็อบเจกต์มีความสามารถในการทำงาน (activities)
 - รถยนต์
 - ขับรถ ส่งของ เดินน้ำมัน
- การกำหนดกิจกรรมให้กับอ็อบเจกต์คือการกำหนดพฤติกรรม (behavior) หรือ ฟังก์ชัน (function) ของอ็อบเจกต์

กำหนดกรอบแนวคิด

- การกำหนดกรอบแนวคิด (concept) ให้กับอ็อบเจกต์
- กำหนดกรอบแนวคิดให้กับสุนัข
 - กำหนดได้ สุนัข มี 4 ขา มี 2 หู มี 1 หาง มีสีขน และมีเสียงทำเสียงโ으่ง
 - ขา ตา หาง สีขน และเสียงโ่่งก็คือลักษณะประจำหรือแอตทริบิวต์ (attribute) ของสุนัข
 - หลังจากที่เราได้กำหนดกรอบแนวคิดให้กับสุนัข
 - ถ้ามีสัตว์ที่มีลักษณะประจำของสุนัขที่ได้นิยามไว้ เราก็จะเรียกสัตว์นั้นว่า สุนัข
 - เช่น คิตteh เป็น ปีกัง และบางแก้ว ก็ถือว่าเป็นสุนัข

คลาส

- การกำหนดกรอบแนวคิดให้กับอ็อบเจกต์ก็คือการสร้างคลาส (class)
- class หรือ abstract object
- คัดลอกเขียน ปีกัง และบางแก้ว เป็นอ็อบเจกต์ของคลาส Dog
- คัดลอกเขียน ปีกัง และบางแก้ว เป็นอินสแตนซ์ (instance) ของคลาส Dog



คลาส
(class หรือ abstract object)

- กำหนดชื่อให้กับคลาส
 - เช่น Dog Cat Student Car
- ยกตัวอย่างชื่อบนจากตัวอย่างคลาสได้
 - เช่น Toyota, Honda และ Ford เป็นชื่อบนจากตัวอย่างชื่อบนของคลาส Car
- สามารถเพิ่มชื่อบนจากตัวอย่างไปให้กับคลาสได้
 - เช่น เช่น Hyundai ให้เป็นชื่อบนจากตัวอย่างคลาส Car

Data type

- คุณลักษณะในการสร้างคลาสที่กล่าวมานี้ก็คือคุณลักษณะของแบบชนิดข้อมูล (data type)
- เช่น int, double, float
 - int ก็คือชื่อของแบบชนิดข้อมูล
 - สามารถยกตัวอย่างของ int ได้ นั่นคือสามารถหาชื่อบนของ int ได้ เช่น int a, b, c;
 - สามารถเพิ่มชื่อบนจากตัวอย่างไปให้กับ int a, b, c, d, e, f;

ลักษณะสำคัญของการเขียนโปรแกรมเชิงอ็อบเจกต์

- การกำหนดสาระสำคัญ (Abstraction)
- การซ่อนข้อมูล (Encapsulation/Information hiding)
- การมีลำดับชั้น (Hierarchy)
- การทำหลายรูปแบบ (Polymorphism)

การกำหนดสาระสำคัญ
Abstraction

- การกำหนดกรอบแนวคิดให้กับชื่อบนจากตัวอย่างเพื่อสร้างคลาส
- คลาสมีความสามารถในการแสดงสิ่งที่ยังซ่อนอยู่ในรูปแบบที่ง่ายต่อการเข้าใจ
 - สร้างอินเตอร์เฟซ (interface) ให้กับคลาส
 - แยกส่วนของอินเตอร์เฟซ (implementation) ออกจากส่วนของอินเตอร์เฟซ

ภาษา C กรณีใช้ Abstraction

โปรแกรมที่ 1.1

```
#include<stdio.h>
void main() {
    int stackData[10];
    int top=0;
    int x, y;

    x = 1;
    stackData[top++] = x; // push
    x = 2;
    stackData[top++] = x; // push
    y = stackData[--top]; // pop
}
```

ภาษา C กรณีใช้ Abstraction

โปรแกรมที่ 1.2

```
#include<stdio.h>

void init(Stack *s);
void push(Stack *s, int i);
int pop(Stack *s);

typedef struct {
    int items[10];
    int top;
} Stack;
```

ภาษา C กรณีใช้ Abstraction

```
void init(Stack *s) {
    s->top=0;
}

void push(Stack *s, int i) {
    s->items[s->top++] = i;
}

int pop(Stack *s) {
    return s->items[--s->top];
}

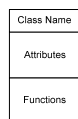
void main() {
    int y;
    Stack myStack;
    init(&myStack);
    push(&myStack, 1);
    push(&myStack, 2);
    y = pop(&myStack);
}
```

Abstract data type (class)

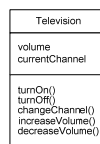
- Data type
 - Description
 - Instance
 - Extension
- Abstract data type
 - data type + attributes + relationship + function

Abstract data type

- **Attributes**
 - ทุกสิ่งในโลกมีส่วนมีลักษณะเฉพาะ (characteristic)
 - ex: height, width, name, location
- **Relationship**
 - Association relationship
 - 1-1, 1-m, m-m
 - Husband-wife, mother-child, brother-sister, teacher-project
 - Generalization relationship
 - Aggregation relationship
 - Composition relationship
- **Function**
 - Behavior, operation, method



Abstract data type (class)



คลาส Television

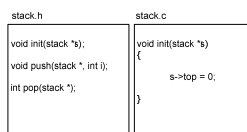
ลักษณะสำคัญของการเขียนโปรแกรมเชิงอ็อบเจกต์

- การกำหนดสาระสำคัญ (Abstraction)
- การห่อหุ้ม/การซ่อนสารสนเทศ (Encapsulation/Information hiding)
- การมีลำดับชั้น (Hierarchy)
- การทำหลายรูปแบบ (Polymorphism)

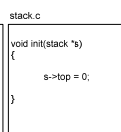
การห่อหุ้ม/การซ่อนสารสนเทศ

- เป็นกลไกในการซ่อนสิ่งที่ซับซ้อนเอาไว้ โดยสิ่งที่ซับซ้อนนั้นจะถูกแสดงให้อยู่ในรูปแบบที่ง่ายต่อการเข้าใจ

ภาษา C



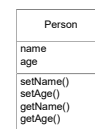
User ต้น



User ไม่ต้น

ภาษา C ทำการซ่อนรายละเอียดของการพิมพ์ลิแวนต์และซ่อนเอาไว้ในอีกไฟล์หนึ่งซึ่งผู้ใช้ไม่ต้น แต่ใน C++ สามารถซ่อนส่วนของฟังก์ชันลิแวนต์และซ่อนไว้ในคลาสได้

คลาส Person และเขียนเจกต์ของคลาส Person

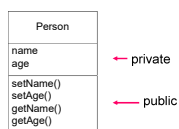


อ็อบเจกต์ a



อ็อบเจกต์ b

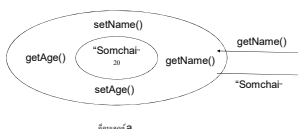
Encapsulation/Information hiding



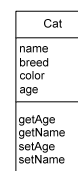
private

public

การห่อหุ้มข้อมูลของอ็อบเจกต์ a



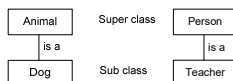
อ็อบเจกต์ a



ลักษณะสำคัญของการเขียนโปรแกรมเชิงอ็อบเจกต์

- การกำหนดสาระสำคัญ (Abstraction)
- การซ่อนข้อมูล (Encapsulation/Information hiding)
- การมีลำดับชั้น (Hierarchy)
- การทำหลายรูปแบบ (Polymorphism)

การมีลำดับชั้น (Hierarchy)



- การรับทอด (inheritance) เป็นความสัมพันธ์ระหว่างคลาสแบบตามลำดับชั้น
- คลาสหนึ่งสามารถรับทอดคุณสมบัติมาจากอีกคลาสหนึ่งได้
- คุณสมบัติที่รับทอดมาคือแอตทริบิวต์และฟังก์ชัน

การมีลำดับชั้น (Hierarchy)



- คลาสที่ทำหน้าที่ถ่ายทอดคุณสมบัติ เรียกว่า คลาสแม่ (parent class) หรือ คลาสพื้นฐาน (base class) หรือซูเปอร์คลาส (superclass)
- คลาสที่รับทอดคุณสมบัติมาจากคลาสแม่ เรียกว่า คลาสลูก (child class) หรือคลาสดิสนิต (derived class) หรือซับคลาส (subclass)
- คลาสลูกนั้นนอกจากจะมีคุณสมบัติแบบเดียวกับของคลาสแม่แล้วยังสามารถมีคุณสมบัติอื่นๆ เพิ่มเติมได้ด้วย

ตัวอย่าง

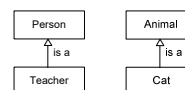
- “สัตว์น้ำ” คือสัตว์ที่อาศัยอยู่ในน้ำ
- “ปลา” เป็นสัตว์น้ำชนิดหนึ่ง
- ปลาและสัตว์น้ำ เป็นความสัมพันธ์แบบการรับทอด
 - ปลาเป็นคุณสมบัติคือเป็นสัตว์ที่อาศัยอยู่ในน้ำ และมีคุณสมบัติเพิ่มเติมคือ
 - มีครีบ หาง ลำตัวแบนๆ และมีกระดูกสันหลัง เป็นต้น



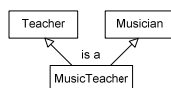
Hierarchy

- ความสัมพันธ์แบบ น้อยทั่วไป/น้อยเฉพาะ (generalization/specialization)
- คลาสลูกมีคุณสมบัติที่เฉพาะเจาะจงและรายละเอียดลงไปมากกว่าคุณสมบัติของคลาสแม่
- คลาสแม่มีคุณสมบัติทั่วไปโดยไม่ได้เฉพาะเจาะจงเมื่อเทียบกับคลาสลูก
- ความสัมพันธ์แบบการรับทอดแบ่งได้เป็นสองประเภท
 - การรับทอดแบบเดี่ยว (single inheritance)
 - การรับทอดแบบหลายแบบ (multiple inheritance)

การรับทอดแบบเดี่ยว single inheritance



การรับทอดแบบหลายแบบ multiple inheritance



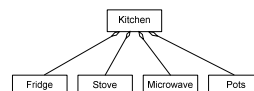
Association

- ความสัมพันธ์ระหว่างคลาสอาจอยู่ในรูปแบบหนึ่งต่อหนึ่ง (1-1) หรือกลุ่ม (1-m) กลุ่มต่อหนึ่ง (m-1) หรือ กลุ่มต่อกลุ่ม (m-m)
- คลาสเป็นอิสระต่อกัน สามารถทำงานแยกจากกันได้
- คลาสมาเกี่ยวข้องกันซึ่งคลาสหนึ่งสามารถเรียกใช้แอตทริบิวต์และฟังก์ชันของอีกคลาสนั้นได้
- เป็นความสัมพันธ์แบบ “has a”



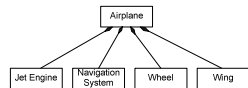
Aggregation

- เป็นความสัมพันธ์รูปแบบหนึ่งของ Association
- เป็นความสัมพันธ์ระหว่างคลาสน้อยหลายๆ คลาสมารวมกันเป็นคลาสน้อย
- คลาสน้อยเหล่านี้มีความคิดแตกต่างกัน แต่เมื่อนำมารวมกันแล้วทำให้เกิดแนวความคิดใหม่ขึ้นมา นั่นคือทำให้เกิดคลาสน้อยขึ้นมา
- เป็นความสัมพันธ์แบบ “part of”

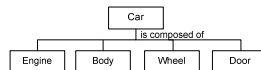


Composition

- เป็นความสัมพันธ์รูปแบบหนึ่งของ **Association**
- มีลักษณะเดียวกับ **Aggregation** แต่แตกต่างกันที่ช่วงชีวิตของส่วนย่อยนั้นจะต้องถูกควบคุมโดยส่วนรวม
- ส่วนย่อยและส่วนรวมไม่สามารถแยกออกจากกันได้ นั่นคือส่วนย่อยจะอยู่เดี่ยวๆ โดยไม่มีส่วนรวมไม่ได้
- ส่วนรวมจะทำหน้าที่สร้างและทำลายส่วนย่อย



Composition



ลักษณะสำคัญของการเขียนโปรแกรมเชิงอ็อบเจกต์

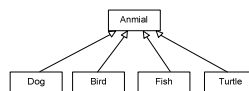
- การกำหนดสาระสำคัญ (**Abstraction**)
- การซ่อน/การซ่อนสารสนเทศ (**Encapsulation/Information hiding**)
- การมีลำดับชั้น (**Hierarchy**)
- การทำหลายรูปแบบ (**Polymorphism**)

Polymorphism

- การทำหลายรูปแบบ หรือ โพลิมอร์ฟิซึม
- หมายถึงการที่ฟังก์ชันหนึ่งสามารถกระทำได้หลายรูปแบบ
 - overloading
 - overriding
 - dynamic binding

Polymorphism

- เราสามารถส่งข้อความชุดเดียวกันไปยังอ็อบเจกต์ต่างๆ โดยที่ไม่จำเป็นต้องทราบว่าอ็อบเจกต์ที่เราส่งข้อความไปนั้นเป็นอ็อบเจกต์ของคลาสใด และอ็อบเจกต์เหล่านั้นก็สามารถตอบสนองต่อข้อความชุดเดียวกันนี้ได้แตกต่างกัน โดยที่มันจะขึ้นอยู่กับอ็อบเจกต์เหล่านั้นจะมีความสัมพันธ์กับแบบการรับทอด
- เช่น **move** ของแต่ละคลาสจะทำหน้าที่ไม่เหมือนกัน



Polymorphism

- **Dynamic binding:** (Late binding); the types of all variables and expressions are not known until runtime.
 - Static binding: the type of all variables and expressions are fixed at the time of compilation.

The key characteristics of an Object-Oriented Method

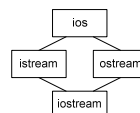
1. Abstraction
2. Encapsulation/Information hiding
3. Hierarchy
4. Polymorphism
 - Object-based language (1,2,3)
 - Object-oriented language (1,2,3,4)

Structure of C++

```

#include<iostream>
using namespace std;
int main() {
    cout<<"hello, world\n";
}
  
```

operator overloading



```

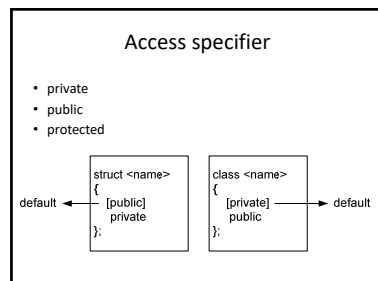
#include<iostream>
using namespace std;
int main() {
    cout << 22.0/7.0 << endl;
    cout << 'a' << endl;
    cout << (4&3) << endl;
    cout << (4&5) << endl;
}
  
```

```

1: 14286
2: a
3: 4
4: 5
  
```

```

4 → 100
5 → 101 &
    100
  
```



COS2102

บทที่ 2

คลาสและออบเจกต์
(Class and Object)

โปรแกรมที่ 2.1

```
#include<iostream>
using namespace std;
class Box {
private:
    int width;
    int height;
    int length;
public:
    void set(int w, int h, int l) {
        width=w;
        height=h;
        length=l;
    }
    int getWidth() {
        return width;
    }
    int getHeight() {
        return height;
    }
    int getLength() {
        return length;
    }
    void show(void);
};

void Box::show() {
    cout<<"Width = "<<width<<endl;
    cout<<"Height = "<<height<<endl;
    cout<<"Length = "<<length<<endl;
    cout<<endl;
}

void main() {
    Box myBox, yourBox;
    myBox.set(4, 5, 6);
    yourBox.set(8, 6, 10);
    myBox.show();
    yourBox.show();

    int volume;
    volume=myBox.getWidth()*myBox.getHeight()*myBox.getLength();
    cout<<"myBox volume = "<<volume<<endl;
}

// inline function
// Declare non-inline function
```

inline function

- คอมไพเลอร์จะนำฟังก์ชันไปใช้ทุกที่ที่ฟังก์ชันนั้นถูกเรียกใช้
 - เพื่อลดเวลาในการเรียกใช้ฟังก์ชัน
 - มักใช้กับฟังก์ชันที่ถูกเรียกใช้บ่อยๆ
 - ใช้กับฟังก์ชันเล็กๆ ที่ไม่ซับซ้อน เช่น ถ้ามี for, while ก็ไม่ควรทำเป็น inline function
- กรณีที่มีนามฟังก์ชันไม่ซับซ้อนขนาด จะต้องใช้ตัว inline


```
inline void Box::Show() {
    cout<<"Width = "<<width<<endl;
    cout<<"Height = "<<height<<endl;
    cout<<"Length = "<<length<<endl;
    cout<<endl;
}
```

โปรแกรมที่ 2.1

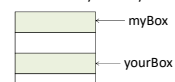
```
#include<iostream>
using namespace std;
class Box {
private:
    int width;
    int height;
    int length;
public:
    void set(int w, int h, int l) {
        width=w;
        height=h;
        length=l;
    }
    int getWidth() {
        return width;
    }
    int getHeight() {
        return height;
    }
    int getLength() {
        return length;
    }
    void show(void);
};

void Box::show() {
    cout<<"Width = "<<width<<endl;
    cout<<"Height = "<<height<<endl;
    cout<<"Length = "<<length<<endl;
    cout<<endl;
}
```

```
void main() {
    Box myBox, yourBox;
    myBox.set(4, 5, 6);
    yourBox.set(8, 6, 10);
    myBox.show();
    yourBox.show();

    int volume;
    volume=myBox.getWidth()*myBox.getHeight()*myBox.getLength();
    cout<<"myBox volume = "<<volume<<endl;
}
```

มีการจองพื้นที่ให้กับ myBox และ yourBox

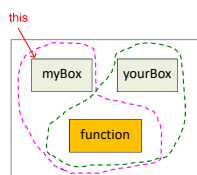


ขนาดของ myBox ?

ขนาดของ yourBox ?

```
void main() {
    Box myBox, yourBox;
    myBox.set(4, 5, 6);
    yourBox.set(8, 6, 10);
    myBox.show();
    yourBox.show();

    int volume;
    volume=myBox.getWidth()*myBox.getHeight()*myBox.getLength();
    cout<<"myBox volume = "<<volume<<endl;
}
```

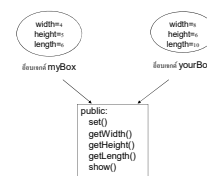
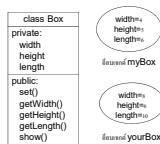


Box *this;

```
class Box {
private:
    int width;
    int height;
    int length;
public:
    ...
    sizeof(int)=?
```

```
void main() {
    Box myBox, yourBox;
    myBox.set(4, 5, 6);
    yourBox.set(8, 6, 10);
    myBox.show();
    yourBox.show();

    int volume;
    volume=myBox.getWidth()*myBox.getHeight()*myBox.getLength();
    cout<<"myBox volume = "<<volume<<endl;
}
```



```

void main() {
    Box myBox, yourBox;
    myBox.set(4, 5, 6);
    yourBox.set(8, 6, 10);
    myBox.show();
    yourBox.show();

    int volume;
    volume=myBox.getWidth()*myBox.getHeight()*myBox.getLength();
    cout<<"myBox volume = "<<volume<<endl;
}

volume=myBox.width*myBox.height*myBox.length;  มีค

```

ผลลัพธ์โปรแกรม

Width = 4
Height = 5
Length = 6

Width = 8
Height = 6
Length = 10

myBox volume = 120

คอนสตรัคเตอร์ Constructor

- คอนสตรัคเตอร์จะมีชื่อเดียวกับชื่อของคลาส
- คอนสตรัคเตอร์จะไม่มีการคืนค่าใดๆ
- ไม่สามารถใส่ค่าใดๆไว้ได้ๆ นำนั่นคอนสตรัคเตอร์ได้
- เนื่องจากคอนสตรัคเตอร์จะถูกเรียกใช้เมื่อมีการสร้างช็อบเบกต์ขึ้นมาใหม่ ดังนั้นคอนสตรัคเตอร์จึงต้องอยู่ในส่วน **public** ของคลาส

โปรแกรมที่ 2.2

```

#include<iostream>
using namespace std;
class Box {
private:
    int width;
    int height;
    int length;
public:
    Box() {
        width=0;
        height=0;
        length=0;
        cout<<"My constructor"<<endl;
    }

    void set(int w, int h, int l) {
        width=w; height=h; length=l;
    }
    int getWidth() { return width; }
    int getHeight() { return height; }
    int getLength() { return length; }
    void show(void);

    void main() {
        Box aBox;
        Box bBox, cBox;
    }
}

ผลลัพธ์โปรแกรม
My constructor
My constructor
My constructor

```

คอนสตรัคเตอร์โอเวอร์โหลดคั้ง Constructor overloading

- คลาสสามารถมีคอนสตรัคเตอร์ได้มากกว่า 1 ฟังก์ชัน
- คอมไพเลอร์สามารถแยกแยะคอนสตรัคเตอร์แต่ละฟังก์ชันได้โดยการตรวจสอบอาร์กิวเมนต์
 - จำนวนอาร์กิวเมนต์
 - ชนิดของอาร์กิวเมนต์

โปรแกรมที่ 2.3

```

#include<iostream>
using namespace std;
class Box {
private:
    int width;
    int height;
    int length;
public:
    Box() { set(0,0,0); }
    Box(int x) { set(x, 0, 0); }
    Box(int x, int y) { set(x, y, 0); }
    Box(int x, int y, int z) { set(x, y, z); }

    void set(int w, int h, int l) {
        width=w; height=h; length=l;
    }
    int getWidth() { return width; }
    int getHeight() { return height; }
    int getLength() { return length; }
    void show(void);
    void addWidth(Box b);

    void Box::addWidth(Box b) {
        int total;
        total = width + b.width;
        cout<<"total="<<total<<endl;
    }
}

Default constructor

```

```

void Box::show() {
    cout<<"Width = "<<width<<endl;
    cout<<"Height = "<<height<<endl;
    cout<<"Length = "<<length<<endl;
    cout<<endl;
}

void main() {
    Box aBox;           → เรียกใช้ฟังก์ชันคอนสตรัคเตอร์ที่มีค่าคือ Box();
    Box bBox(2);        → เรียกใช้ Box(int x);
    Box cBox(3,4);      → เรียกใช้ Box(int x, int y);
    Box dBox(5,6,7);    → เรียกใช้ Box(int x, int y, int z);
    dBox.addWidth(cBox);
}

ผลลัพธ์โปรแกรม
total=8

```

```

void main() {
    int total;
    total = dBox.width + cBox.width;  ?
}

```

- กรณีที่ไม่ได้เขียนคอนสตรัคเตอร์ไว้บนคลาส คอมไพเลอร์จะถือว่าคลาสนั้นมีฟังก์ชันคอนสตรัคเตอร์อยู่ ซึ่งเป็นฟังก์ชันว่างเปล่า ไม่ได้ทำอะไรอะไร

```

class MyExample {
private:
    int myEx;
public:
    void set(int i) { myEx=i; }
    void show() { cout<<"My example is"<<myEx<<endl; }
};

void main() {
    MyExample a;
}

```

```

class MyExample {
private:
    int myEx;
public:
    MyExample(int j) { set(j); }
    void set(int i) { myEx=i; }
    void show() { cout<<"My example is"<<myEx<<endl; }
};

void main() {
    MyExample a;      error
    MyExample b(5);   ok
}

```

- นอกจากนี้ฟังก์ชันคอนสตรัคเตอร์ที่คอมไพเลอร์จะทำการสร้างให้โดยอัตโนมัติแล้ว ยังมีอีกสามฟังก์ชันที่คอมไพเลอร์จะสร้างให้ในกรณีที่ไม่ได้มีการสร้างเอาไว้ในคลาสคือ
 - ฟังก์ชันที่เรียกใช้คอนสตรัคเตอร์ (default copy constructor)
 - ฟังก์ชันที่สตรัคเตอร์ (default destructor)
 - โอเปอเรเตอร์ที่ทำงานคล้ายกับตัวคูณ (default assignment operator)
- ตัวอย่างการเรียกใช้ copy constructor


```
Box gBox(eBox);
Box hBox = eBox;
```
- ตัวอย่างการเรียกใช้ assignment operator


```
hBox = eBox;
```

โปรแกรมที่ 2.4

```
#include<iostream>
using namespace std;
class Box {
    int width;
    int height;
    int length;
public:
    Box() : width(0), height(0), length(0) {}
    Box(int x) : width(x), height(0), length(0) {}
    Box(int x, int y) : width(x), height(y), length(0) {}
    Box(int x, int y, int z) : width(x), height(y), length(z) {}
    void show(void);
};
```

```
void main() {
    Box aBox;           Width = 0
    aBox.show();         Height = 0
    Box bBox(2);         Length = 0
    bBox.show();         Width = 2
    Box cBox(3,4);       Height = 0
    cBox.show();         Length = 0
    Box dBox(5,6,7);     Width = 3
    dBox.show();         Height = 4
                        Length = 0
    Width = 5
    Height = 6
    Length = 7
}
```

โปรแกรมที่ 2.5 ใช้ค่าที่ฟังก์ชันการนิยาม

```
#include<iostream>
using namespace std;
class Box {
    int width;
    int height;
    int length;
public:
    Box(int x=0, int y=0, int z=0) : width(x), height(y), length(z) {}
    void show(void);
};
```

ดีสตรัคเตอร์

- เมื่อเขียนจบจากฟังก์ชันการนิยามจะมีการเรียกใช้ดีสตรัคเตอร์โดยอัตโนมัติ
- ดีสตรัคเตอร์จะมีชื่อเดียวกับคอนสตรัคเตอร์
- ไม่มีการคืนค่าใดๆ
- ไม่มีอาร์กิวเมนต์
- ไม่มีการโอเวอร์โหลดดีสตรัคเตอร์
- คลาสจะมีดีสตรัคเตอร์ได้เพียงหนึ่งฟังก์ชันเท่านั้น
- เขียนสัญลักษณ์ “~” ไว้ข้างหน้าดีสตรัคเตอร์

โปรแกรมที่ 2.6

```
#include<iostream>
using namespace std;
class MyName {
    char name[10];
public:
    MyName() {
        strcpy(name, "noname");
        cout<<"Hi: "<<name<<endl;
    }
    MyName(char *n) {
        strcpy(name, n);
        cout<<"Hi: "<<name<<endl;
    }
    ~MyName() {
        cout<<"bye bye: "<<name<<endl;
    }
};

void main() {
    MyName n1;
    MyName n2("Somchai");
    {
        MyName n3("Somkid");
    }
    MyName n4("Somrak");
    //ผลลัพธ์โปรแกรม
    Hi: noname
    Hi: Somchai
    Hi: Somkid
    bye bye: Somkid
    Hi: Somrak
    bye bye: Somrak
    bye bye: Somchai
    bye bye: noname
}
```

```
class A
{
    int x;
public:
    A() { x=0; cout << x << endl; }
    A(int i) { x=i; cout << x << endl; }
};
class B
{
    int y;
public:
    A k;
    B() { y=0; cout << y << endl; }
    B(int i) { y=i; cout << y << endl; }
};

void main()
{
    A a;
    B b;
    A c(5);
    B d(10);
    0
    0
    0
    5
    0
    10
}
```

อ็อบเจกต์คงที่

```
const int x=25;
void f(char *a, const int b);
```

- x และ b เป็นค่าคงที่
- ไม่สามารถแก้ไขค่า x และ b ได้

```
const Box myBox(3,4,5);
```

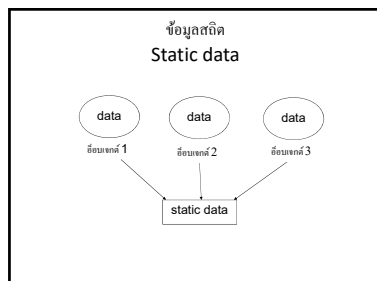
- อ็อบเจกต์คงที่สามารถเรียกใช้คอนสตรัคเตอร์และดีสตรัคเตอร์ได้
- แต่ไม่สามารถเรียกใช้ฟังก์ชันปกติได้
- ดังนั้น myBox จึงไม่สามารถเรียกใช้ฟังก์ชัน show() ได้

- ถ้าต้องการให้อ็อบเจกต์คงที่เรียกใช้ฟังก์ชันได้
- ฟังก์ชันนั้นต้องเป็นค่าคงที่
- และต้องเป็นฟังก์ชันที่ไม่นำไปแก้ไขค่าของตัวแปร
- ถ้าต้องการให้ myBox เรียกใช้ฟังก์ชัน show() จะต้องทำฟังก์ชัน const show() ให้เป็นค่าคงที่โดยการใส่คีย์เวิร์ด const

```
void show(void) const;
```

- ถ้านิยามฟังก์ชันนี้ภายนอกคลาสก็จะต้องใส่คีย์เวิร์ด const ด้วย

```
void Box::show() const {
    cout<<"Width = "<<width<<endl;
    cout<<"Height = "<<height<<endl;
    cout<<"Length = "<<length<<endl;
    cout<<endl;
}
```



```

#include <iostream>
using namespace std;

int x;

int f1() {
    return ++x;
}

int f2() {
    int y=x;
    return ++y;
}

int f3() {
    static int z;
    return ++z;
}

int main() {
    cout<<f1()<<endl;
    cout<<f2()<<endl;
    ++x;
    cout<<f1()<<endl;
    cout<<f2()<<endl;
    cout<<f3()<<endl;
    cout<<f2()<<endl;
    cout<<f3()<<endl;
    cout<<f3()<<endl;
    return 0;
}
  
```

โปรแกรมที่ 2.7

```

#include <iostream>
using namespace std;

class MyObj {
private:
    static int num;
    int width;
    int height;
    float price;
public:
    MyObj() {
        num++;
        set(0,0,0.0);
        cout<<num<<endl;
    }
    MyObj(int w, int h) {
        num++;
        set(w, h, 0.0);
        cout<<num<<endl;
    }
    MyObj(float p) {
        num++;
        set(0, 0, p);
        cout<<num<<endl;
    }
    MyObj(int w, int h, float p) {
        num++;
        set(w, h, p);
        cout<<num<<endl;
    }
}
  
```

```

~MyObj() {
    num--;
    cout<<num<<endl;
}

void set(int w, int h, float p) {
    width=w; height=h; price=p;
}

int MyObj::num;
void main() {
    MyObj book;
    {
        MyObj pencil(1, 14), table(50, 60, 1500.0);
    }
    MyObj mobile(4, 10, 15000.0);
}
  
```

ผลลัพธ์โปรแกรม

```

1
2
3
2
1
2
1
0
  
```

ฟังก์ชันสถิต
Static function

- ฟังก์ชันสถิตไม่ใช่สมาชิกของคลาส
- สามารถเรียกใช้ฟังก์ชันนี้ได้ถึงแม้ว่าจะไม่มีชื่อเบจกต์ใดๆ ถูกสร้างขึ้นเลยในโปรแกรม
- ฟังก์ชันสถิตสามารถเข้าถึงข้อมูลสถิตจากคลาสที่มีมันเป็นเจ้าของได้เท่านั้น

โปรแกรมที่ 2.8

```

#include <iostream>
using namespace std;
class MyObj {
private:
    static int num;
    int width;
    int height;
    float price;
public:
    MyObj() {
        num++; set(0, 0, 0.0);
    }
    MyObj(int w, int h, float p) {
        num++; set(w, h, p);
    }
}

~MyObj() {
    num--;
    cout<<num<<endl;
}

void set(int w, int h, float p) {
    width=w; height=h; price=p;
}

static int count() { return num; }

int MyObj::num;
  
```

```

void main() {
    cout<<MyObj::count()<<endl;
    MyObj book;
    cout<<MyObj::count()<<endl;
    {
        MyObj pencil(1, 14, 20.0), table(50, 60, 1500.0);
        cout<<MyObj::count()<<endl;
    }
    MyObj mobile(4, 10, 15000.0);
    cout<<MyObj::count()<<endl;
}
  
```

ผลลัพธ์โปรแกรม

```

0
1
3
2
1
2
1
0
  
```

```

class Foo {
public:
    class Bar { /* ... */ }
    typedef int myInt;
    enum {RED, BLUE, GREEN};
    static int staticDataMem;
    static void staticMemFunc() {cout<<"test";}
};

int Foo::staticDataMem;

void main() {
    Foo::myInt myColor = Foo::BLUE;
    Foo::staticDataMem=5;
    cout<<myColor;
    Foo::staticMemFunc();
}
  
```

1test

```

class Foo {
    int privData1;
    void privFunc1;
public:
    int pubData;
    void pubFunc1();
private:
    void privFunc2();
private:
    double privData2;
public:
    void pubFunc3();
};
  
```

COS2102

บทที่ 3
 อ็อบเจกต์ไดนามิกและอาร์เรย์ไดนามิก
 Dynamic Object and
 Dynamic Array of Objects

โปรแกรมที่ 3.1 อารีเวย์ไดนามิก

```
#include<iostream>
using namespace std;
class MyObj {
    int width; int height; float price;
public:
    void setObj();
    void showObj();
};
void MyObj::setObj() {
    cout<<"Enter width: "; cin>>width;
    cout<<"Enter height: "; cin>>height;
    cout<<"Enter price: "; cin>>price;
}
void MyObj::showObj() {
    cout << "Width = "<< width << endl;
    cout << "Height = "<< height << endl;
    cout << "Price = "<< price << endl;
}
```

```
void main() {
    int max;
    cout<<"How many objects?"<<endl;
    cin>>max;
    MyObj mobile[max];
    for (int i=0; i<max; i++) {
        cout<<"Object no."<<i+1<<endl;
        mobile[i].setObj();
    }
    cout<<"--Show results--"<<endl;
    for (int j=0; j<max; j++) {
        cout<<"Show object no."<<j+1;
        cout<<endl;
        mobile[j].showObj();
    }
}
```

ผลลัพธ์โปรแกรม

How many objects?
 2
 Object no.1
 Enter width: 4
 Enter height: 10
 Enter price: 2500
 Object no.2
 Enter width: 5
 Enter height: 12
 Enter price: 5000
 --Show results--
 Show object no.1
 Width = 4
 Height = 10
 Price = 2500
 Show object no.2
 Width = 5
 Height = 12
 Price = 5000

โปรแกรมที่ 3.2 การเรียกใช้คอนสแตนต์และตัวชี้ตัวแปรของอ็อบเจกต์ที่ไม่อยู่ในหน่วย

```
#include<iostream>
using namespace std;
class MyName {
    char name[10];
public:
    MyName() {
        strcpy(name, "noname");
        cout<<"Hi "<<name<<endl;
    }
    MyName(char *n) {
        strcpy(name, n);
        cout<<"Hi "<<name<<endl;
    }
    ~MyName() {
        cout<<"bye bye "<<name<<endl;
    }
};
```

```
void main() {
    MyName n[2];
    MyName p[3]={"Somkid", "Pawalai", "Sathit"};
    MyName r[3][2]={"r1", "r2", "r3", "r4", "r5", "r6"};
}
```

ผลลัพธ์โปรแกรม

```
Hi noname      bye bye r6
Hi noname      bye bye r5
Hi Somkid      bye bye r4
Hi Pawalai     bye bye r3
Hi Sathit      bye bye r2
Hi r1          bye bye r1
Hi r2          bye bye Sathit
Hi r3          bye bye Pawalai
Hi r4          bye bye Somkid
Hi r5          bye bye noname
Hi r6          bye bye noname
```

```
void main()
{
    MyName m;
    MyName n[2];
    MyName *p;
}
```

p เป็น object ? [ไม่เป็น](#)

```
int x, y;
int *p;
x = 1;
p = &x;
y = *p;      y=1
*p = *p+1;   x=2
y = ++*p;    y=3, x=3
y = (*p)++;  y=3, x=4
y = *p++;    y=4, p ชี้ถัดไป
```

มีสองวิธีในการจองพื้นที่

- Dynamic objects
 - ใช้ new และ delete
- Dynamic arrays of objects
 - ใช้ new[] และ delete[]

อ็อบเจกต์ไดนามิก
dynamic object

```
MyObj *radio;
radio = new MyObj;
...
delete radio;
```

หรือ

```
MyObj *radio = new MyObj;
...
delete radio;
```

radio จะเป็น object หลังจากที่มีการ allocate memory แล้ว

อ็อบเจกต์พลวัต

- **new**
 - จัดพื้นที่หน่วยความจำให้กับอ็อบเจกต์และเรียกใช้คอนสตรัคเตอร์
 - จะได้พอยเตอร์ที่มีหน่วยความจำที่เพียงพอสำหรับใช้งาน
- **delete**
 - เรียกใช้ดีสคอนสตรัคเตอร์และคืนหน่วยความจำให้กับระบบ

โปรแกรมที่ 3.3

```
#include<iostream>
using namespace std;
class MyObj {
    int width;
    int height;
    float price;
public:
    MyObj() { cout<<"my object\n"; }
    MyObj(int w, int h, float p) {
        set(w, h, p);
        cout<<"Obj: "<<width<<" "<<height<<" "<<price<<endl;
    }
    ~MyObj() { cout<<"bye bye\n"; }
    void set(int w, int h, float p) {
        width=w, height=h, price=p;
    }
};
```

```
void main()
{
    MyObj *t;
    cout<<"hello\n";

    MyObj *p = new MyObj;
    p->set(4, 10, 980.0);
    (*p).set(5, 10, 12000.0);
    delete p;

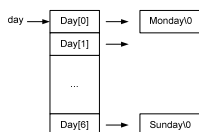
    p = new MyObj(5, 20, 550.50);
    delete p;
}
```

ผลลัพธ์โปรแกรม

```
hello
my object
bye bye
Obj: 5 20 550
bye bye
```

Array of pointers

- `char *p[10];`
- `char *p[10][10];`
- `char *day[] = {"Monday", ..., "Sunday"};`



Pointer to array

- `char (*p)[10];`
- `char p[][10];`
- `char day[7][10] = {"Monday", ..., "Sunday"};`
`= { {'M','o','n','d','a','y','\0','\0','\0','\0'},`
`{ 'T',..., },`
`...,`
`};`

โปรแกรมที่ 3.4

```
#include<iostream>
using namespace std;
class MyObj {
    int width; int height; float price;
public:
    MyObj() { set(0,0,0); }
    MyObj(int w, int h, float p) { set(w,h,p); }
    void set(int w, int h, float p) {
        width=w; height=h; price=p;
    }
    void show() { cout<<width<<" "<<height<<" "<<price<<endl; }
    ~MyObj() { cout<<"bye bye\n"; }
};
```

```
void main() {
    MyObj *y[3] = { new MyObj,
                    new MyObj(2,3,100.5),
                    new MyObj(5,6,750.0) };
}
```

```
for (int i=0; i<3; i++)
    y[i]->show();
```

```
for (int i=0; i<3; i++)
    delete y[i];
```

ผลลัพธ์โปรแกรม

```
0 0 0
2 3 100.5
5 6 750
bye bye
bye bye
bye bye
```

อาร์เรย์พลวัต Dynamic arrays of objects

```
MyObj *q = new MyObj[5];
...
delete [] q;
```

โปรแกรมที่ 3.5

```
#include<iostream>
using namespace std;
class MyIntArray {
    int s;
    int *a;
public:
    MyIntArray() { s=0; a=0; }
    MyIntArray(int y) { set(y); }
    ~MyIntArray() {
        delete [] a;
        cout<<"bye bye\n";
    }
    void set(int y) { s=y; a = new int[s]; }
    void setA();
    void showA();
    void reverse();
};
```

```
void MyIntArray::setA() {
    cout<<"Enter values"<<endl;
    for (int i=0; i<s; i++) {
        cout<<"a["<<i<<"]="<<endl;
        cin>>a[i];
    }
}

void MyIntArray::showA() {
    cout<<"Show values"<<endl;
    for (int i=0; i<s; i++) {
        cout<<"a["<<i<<"]="<<a[i]<<endl;
    }
}

void MyIntArray::reverse() {
    int temp;
    for (int i=0; i<s/2; i++) {
        temp = a[i];
        a[i] = a[s-i-1];
        a[s-i-1] = temp;
    }
}
```


COS2102

บทที่ 4

การอ้างอิง

Reference

โปรแกรมที่ 4.1

```
#include<iostream>
using namespace std;
void main() {
    int n=5;
    cout<<"n = "<<n<<" r = "<<r<<endl;
    if (&r == &n)
        cout<<"Hello"<<endl;
}
```

ผลลัพธ์โปรแกรม

```
n = 5 r = 5
Hello
```

โปรแกรมที่ 4.2

ผลลัพธ์โปรแกรม

```
#include<iostream>
using namespace std;
void main() {
    int n=5;
    int &r=n;
    int &k=r;
    cout<<"n = "<<n<<" r = "<<r<<" k = "<<k<<" i = "<<i<<endl;
    n++;
    cout<<"n = "<<n<<" r = "<<r<<" k = "<<k<<" i = "<<i<<endl;
    r++;
    cout<<"n = "<<n<<" r = "<<r<<" k = "<<k<<" i = "<<i<<endl;
    k++;
    cout<<"n = "<<n<<" r = "<<r<<" k = "<<k<<" i = "<<i<<endl;
    i++;
    cout<<"n = "<<n<<" r = "<<r<<" k = "<<k<<" i = "<<i<<endl;
}
```

```
n = 5 r = 5 k = 5 i = 5
n = 6 r = 6 k = 6 i = 6
n = 7 r = 7 k = 7 i = 7
n = 8 r = 8 k = 8 i = 8
n = 9 r = 9 k = 9 i = 9
```

การส่งผ่านค่าของฟังก์ชัน

- การส่งผ่านด้วยตัวแปร (pass by value)
- การส่งผ่านด้วยพอยเตอร์ (pass by pointer)
- การส่งผ่านด้วยการอ้างอิง (pass by reference)

โปรแกรมที่ 4.3

```
#include<iostream>
using namespace std;
void f(int a) {
    cout<<a<<endl; // a is a parameter
}

void main() {
    int i=5;
    f(10); // 10 is an argument
    f(i); // i is an argument
}
```

โปรแกรมที่ 4.4

```
#include<iostream>
using namespace std;
class MyObj {
public:
    int width; int height; float price;
    MyObj() {
        width=height=0;
        price=0;
    }
    void setA(int w, int h) {
        ++w; ++h;
        width=w; height=h;
    }
    void setB(int *a, int *b) {
        ++*a; ++*b;
        width=*a; height=*b;
    }
}
```

```
void setC(int &c, int &d) {
    ++c; ++d;
    width=c; height=d;
}
void show() {
    cout<<"width="<<width<<" height="<<height<<endl;
}

int main() {
    MyObj k;
    int m,n;
    m=3; n=4;
    k.setA(m,n);
    k.show();
    cout<<"m="<<m<<" n="<<n<<endl;
    k.setB(&m,&n);
    k.show();
    cout<<"m="<<m<<" n="<<n<<endl;
    k.setC(m,n);
    k.show();
    cout<<"m="<<m<<" n="<<n<<endl;
}
```

ผลลัพธ์โปรแกรม

```
width=4 height=5
m=3 n=4
width=4 height=5
m=4 n=5
width=5 height=6
m=5 n=6
```

- ไม่สามารถส่งผ่านอาร์กิวเมนต์ด้วยค่าคงที่
k.setC(3, 4); // error
- ไม่สามารถส่งผ่านอาร์กิวเมนต์ที่เป็นตัวแปรคงที่ไปยังพารามิเตอร์อ้างอิง
const int p=3;
const int q=4;
k.setC(p, q); // error

```
void setC(int &c, int &d) {
    ++c; ++d;
    width=c; height=d;
}
```

โปรแกรมที่ 4.5

```
#include<iostream>
using namespace std;
class MyObj {
public:
    int width; int height; float price;
    MyObj(int i, int j, float k) {
        width=i; height=j; price=k;
    }
    ~MyObj() { cout<<"bye bye\n"; }
    float addPriceV(MyObj x) {
        x.price+=10;
        return (price+x.price);
    }
    float addPriceR(MyObj &x) {
        x.price+=10;
        return (price+x.price);
    }
    void showPrice() { cout<<price<<endl; }
};

void main() {
    MyObj m(5, 10, 20.0);
    MyObj n(15, 20, 30.0);
    cout<<m.addPriceV(n)<<endl;
    n.showPrice();
    cout<<endl;
    cout<<m.addPriceR(n)<<endl;
    n.showPrice();
    cout<<endl;
}
```

ผลลัพธ์โปรแกรม

```
60
bye bye
30
60
40
bye bye
bye bye
```

- กรณีที่ไม่ต้องการให้ฟังก์ชันเปลี่ยนแปลงค่าที่ถูกส่งผ่านด้วยการอ้างอิงจะสามารถทำได้โดยการใช้ **const** ในพารามิเตอร์อ้างอิง เพื่อให้ทำไม่เป็นการส่งผ่านตัวแปรอ้างอิงจริง
- วิธีนี้จะทำให้เราสามารถส่งผ่านอาร์กิวเมนต์ด้วยค่าคงที่และตัวแปรคงที่ได้ด้วย

โปรแกรมที่ 4.6

```
#include <iostream>
using namespace std;
class MyObj {
    int width; int height; float price;
public:
    MyObj() {
        width=height=0;
        price=0;
    }
    void set(const int &c, const int &d) {
        width=c; height=d;
        // c=1; d=2; // error
    }
    void show() {
        cout<<"width"<<width<<" height"<<height<<endl;
    }
};
```

```
void main() {
    MyObj k;
    int m,n;
    m=3;
    n=4;
    k.set(m,n);
    k.show();
    k.set(5,6);
    k.show();
    const int p=7;
    const int q=8;
    k.set(p,q);
    k.show();
}
```

ผลลัพธ์โปรแกรม

```
width=3 height=4
width=5 height=6
width=7 height=8
```

โปรแกรมที่ 4.7

แสดงการแตกต่างระหว่างการส่งผ่านตัวแปรโดยค่า และ การส่งผ่านโดยอ้างอิง

```
#include <iostream>
using namespace std;
class MyObj {
    int width; int height;
public:
    MyObj() { width=height=0; }
    void setD(int *p, int *q) {
        **p; **q;
        width=*p; height=*q;
        p = new int(1); q = new int(2);
    }
    void setE(int *r, int *s) {
        **r; **s;
        width=*r; height=*s;
        r = new int(1); s = new int(2);
    }
    void show() {
        cout<<"width"<<width<<" height"<<height<<endl;
    }
};
```

```
void main() {
    MyObj k;
    int *m = new int(3);
    int *n = new int(4);

    cout<<"address before setD():"<<endl;
    cout<<"&m"<<m<<endl;
    cout<<"&n"<<n<<endl<<endl;

    k.setD(m,n);
    k.show();
    cout<<"m"<<" n"<<"<<endl<<endl;

    cout<<"address after setD():"<<endl;
    cout<<"&m"<<m<<endl;
    cout<<"&n"<<n<<endl<<endl;

    cout<<"....."<<endl<<endl;
}
```

ผลลัพธ์โปรแกรม

```
address before setD():
&m=0x3e3bb0
&n=0x3e3c18

width=4 height=5
m=4 n=5

address after setD():
&m=0x3e3bb0
&n=0x3e3c18

.....
```

```
cout<<"address before setE():"<<endl;
cout<<"&m"<<m<<endl;
cout<<"&n"<<n<<endl<<endl;

k.setE(m,n);
k.show();

cout<<"m"<<" n"<<"<<endl;
cout<<endl;
cout<<"address after setE():"<<endl;
cout<<"&m"<<m<<endl;
cout<<"&n"<<n<<endl;
}
```

ผลลัพธ์โปรแกรม

```
address before setE():
&m=0x3e3bb0
&n=0x3e3c18

width=5 height=6
m=1 n=2

address after setE():
&m=0x3e3c48
&n=0x3e3c58
```

การคืนค่าของฟังก์ชัน

- การคืนค่าด้วยค่าของตัวแปร
- การคืนค่าด้วยค่าของพอยเตอร์
- การคืนค่าด้วยค่าของการอ้างอิง

โปรแกรมที่ 4.8

```
#include <iostream>
using namespace std;
int f1() {
    int x;
    x=3;
    return x;
}
int* f2() {
    int *x = new int;
    *x=3;
    return x;
}
int& f3() {
    int x;
    x=3;
    return x;
    // warning: reference to local
    // variable 'x' returned
}
```

```
void main() {
    int p;
    p=f1();
    cout<<p<<endl;

    int *y;
    y=f2();
    cout<<"*y"<<endl;
    delete y;

    int q;
    q=f3();
    cout<<q<<endl;
}
```

ผลลัพธ์โปรแกรม

```
3
3
3
```

คำชี้แจง

- ค่าที่ไม่มีขอบเขตชีวิตในขอบเขตฟังก์ชันจะเป็นค่าซ้าย (value)
- ค่าซ้ายจะหมายถึงค่าที่อยู่ภายใต้การควบคุมโดยหน่วยความจำคอมพิวเตอร์

```
int n, a[3], *p;
const int m = 7;
n = 5;
a[0] = 4;
*p = 6;
```

- n, a[0] และ *p เป็นค่าซ้ายที่สามารถถูกเปลี่ยนแปลงได้
- m เป็นค่าซ้ายที่ไม่สามารถเปลี่ยนแปลงได้

จากโปรแกรมที่ 4.8
f3()=5; ?

โปรแกรมที่ 4.9

```
#include <iostream>
using namespace std;
int y;
int& g() {
    static int x;
    x++;
    return x;
}
int& h() {
    y++;
    return y;
}
void main() {
    g()=5;
    cout<<g()<<endl;
    h()=8;
    cout<<h()<<endl;
}
```

ผลลัพธ์โปรแกรม

```
6
9
```

โปรแกรมที่ 4.10

```
#include<iostream>
using namespace std;
class X {
    int a;
public:
    int &f1(int i) {
        a++;
        return a;
    }
    void show() { cout<<"a"<<a<<endl; }
};
void main() {
    X n;
    n.f1(5)=2;
    n.show();
}
```

ผลลัพธ์โปรแกรม

a=2

จัดเรียงกับหลักการเขียนโปรแกรม

โปรแกรมที่ 4.11

```
#include<iostream>
using namespace std;
class A {
    int a;
    int b;
public:
    A() { a=b=0; }
    A(int i) { a=i; b++; }
    A &f(A &x) { return x; }
    void show() { cout<<"a"<<a<<" b"<<b<<endl; }
};
void main() {
    A n, m;
    n.f(m);
    m.show();
    n.f(m)=5;
    m.show();
}
```

ผลลัพธ์โปรแกรม

a=0 b=0
a=5 b=6
a=0 b=0

```
class Point
{
    int x, y;
public:
    Point() { x=y=0; }
    void reset() { Point(); }
};

void reset() { Point p1; }

Point::init() { x=y=0; }
Point::Point() { init(); }
Point::reset() { init(); }
```

COS2102

บทที่ 5
ความสัมพันธ์ระหว่างคลาส
Class Relationship

- ความสัมพันธ์เชิงเกี่ยวข้อง (association)
- ความสัมพันธ์เชิงรวมกลุ่ม (aggregation)
- ความสัมพันธ์เชิงประกอบ (composition)

Association

- เป็นความสัมพันธ์ระหว่างคลาสในรูปแบบของ "has-a"
- แต่ละอ็อบเจกต์ของคลาสจะมีช่วงชีวิตเป็นของตนเอง ไม่มีความเป็นเจ้าของกันและกัน นั่นคือเป็นอิสระต่อกัน
- แต่ละคลาสนั้นสามารถทำงานแยกจากกันได้
- ความสัมพันธ์ระหว่างคลาสอาจอยู่ในรูปแบบหนึ่งต่อหนึ่ง หนึ่งต่อกลุ่ม กลุ่มต่อหนึ่ง หรือกลุ่มต่อกลุ่ม



โปรแกรมที่ 5.1

```
#include<iostream>
using namespace std;
class Student {
    char sname[20];
public:
    Student() {
        setName("None");
    }
    Student(char *n) {
        setName(n);
    }
    void setName(char* n) {
        strcpy(sname, n);
    }
    char* getName() {
        return sname;
    }
    ~Student() { cout<<sname<<endl; }
};
```

```
class Course {
    char id[20];
public:
    Course() {
        setCourse("COSxxxx");
    }
    Course(char* n) {
        setCourse(n);
    }
    void setCourse(char* n) {
        strcpy(id, n);
    }
    char* getId() { return id; }
    ~Course() { cout<<id<<endl; }
};
```

```
class Enrollment {
    Student *std;
    Course *cos;
    char term[10];
public:
    Enrollment(Student *s, Course *c, char* t): std(s), cos(c) {
        setTerm(t);
        printEnrollment();
    }
    void setTerm(char* n) {
        strcpy(term, n);
    }
    void printEnrollment() {
        cout<<"Name:"<<std->getName()<<
            " Course:"<<cos->getId()<<
            " Term:"<<term<<endl;
    }
};
```

```
void main() {
    Student *s[2] = { new Student("Somkid"), new Student("Panda") };
    Course *c[3] = { new Course("COS2101"),
                    new Course("COS2102"),
                    new Course("COS2103") };
}
```

```
Enrollment e1(s[0], c[0], "1/57");
Enrollment e2(s[0], c[1], "1/57");
Enrollment e3(s[1], c[0], "1/57");
Enrollment e4(s[1], c[1], "2/57");
Enrollment e5(s[1], c[2], "2/57");
```

```
cout<<endl<<"--delete Student--"<<endl;
for (int i=0; i<2; i++)
    delete s[i];

cout<<endl<<"--delete Course--"<<endl;
for (int j=0; j<3; j++)
    delete c[j];
```

ผลลัพธ์โปรแกรม

```
Name: Somkid Course: COS2101 Term: 1/57
Name: Somkid Course: COS2102 Term: 1/57
Name: Panda Course: COS2103 Term: 1/57
Name: Panda Course: COS2102 Term: 2/57
Name: Panda Course: COS2103 Term: 2/57

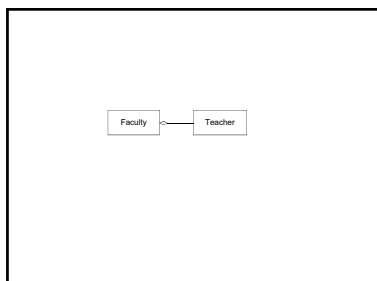
--delete Student--
Somkid
Panda

--delete Course--
COS2101
COS2102
COS2103
```

- อ็อบเจกต์ของ **Student** และ **Course** นั้นเป็นอิสระต่อกัน
- การเกิดและการตายของแต่ละอ็อบเจกต์นั้นไม่ได้มีความเกี่ยวข้องกัน
- แต่ละอ็อบเจกต์มีช่วงชีวิตเป็นของตนเอง
- ไม่มีความเป็นเจ้าของระหว่างอ็อบเจกต์ทั้งสองคลาส

Aggregation

- เป็นรูปแบบหนึ่งของความสัมพันธ์แบบ Association
- เป็นความสัมพันธ์แบบเชิง ส่วนรวม/ส่วนย่อย (whole/part)
- ส่วนรวมจะเป็นคลาสดใหญ่ที่เกิดจากการนำส่วนย่อยหรือคลาสน้อยมารวมกัน
- คลาสน้อยเป็นส่วนหนึ่งของคลาสดส่วนรวม
- เป็นความสัมพันธ์แบบ "part of"
- ส่วนรวมและส่วนย่อยนั้นสามารถแยกออกจากกันแล้วยังทำงานของตัวเองได้ นั่นคือมีช่วงชีวิตเป็นของตนเอง
- ส่วนย่อยจะมีความสัมพันธ์กับส่วนรวมได้เพียงคลาสดเดียวเท่านั้น
- คลาสน้อยจะเป็นส่วนหนึ่งของคลาสดส่วนรวมสองคลาสดหรือนำๆ กันไม่ได้



โปรแกรมที่ 5.2

```

#include<iostream>
using namespace std;
class Teacher {
    char tname[10];
public:
    Teacher() { setName("None"); }
    Teacher(char *n) { setName(n); }
    void setName(char* n) { strcpy(tname, n); }
    char* getName() { return tname; }
    ~Teacher() { cout<<"delete "<<tname<<endl; }
};
  
```

```

class Faculty {
    char fname[20];
    Teacher **listTeacher;
public:
    Faculty() {
        setName("None");
        listTeacher=0;
    }
    Faculty(char *n, Teacher **t) {
        setName(n);
        listTeacher=t;
    }
    void setName(char* n) { strcpy(fname, n); }
    char* getFname() { return fname; }
    char *getTeacher(int i) { return listTeacher[i]->getName(); }
    ~Faculty() { cout<<"delete "<<fname<<endl; }
};
  
```

```

void main() {
    Teacher *t[3] = { new Teacher("Pawalai"),
                     new Teacher("Sathit"),
                     new Teacher("Ratchanee") };
    Faculty *sci = new Faculty("Science", t);

    cout<<"-- Teacher in Faculty --"<<endl;
    for (int i=0; i<3; i++)
        cout<<sci->getTeacher(i)<<endl;

    cout<<"-- delete Faculty --"<<endl;
    delete sci;
    cout<<"-- Teacher still exist --"<<endl;
    for (int i=0; i<3; i++)
        cout<<t[i]->getName()<<endl;

    cout<<"-- delete Teacher --"<<endl;
    for (int i=0; i<3; i++)
        delete t[i];
}
  
```

ผลลัพธ์โปรแกรม

```

-- Teacher in Faculty --
Pawalai
Sathit
Ratchanee
-- delete Faculty --
delete Science
-- Teacher still exist --
Pawalai
Sathit
Ratchanee
-- delete Teacher --
delete Pawalai
delete Sathit
delete Ratchanee
  
```

*** delete listTeacher ***
 delete listTeacher *** delete listTeacher ***
 delete listTeacher *** delete listTeacher ***

Composition

- เป็นความสัมพันธ์รูปแบบหนึ่งของความสัมพันธ์แบบ aggregation
- ช่วงชีวิตของส่วนย่อยจะถูกควบคุมโดยส่วนรวม
- ส่วนรวมทำหน้าที่ควบคุมการเกิดและการตายของส่วนย่อย
- ถ้าส่วนรวมถูกทำลาย จะทำให้ส่วนย่อยถูกทำลายตามไปด้วย



โปรแกรมที่ 5.3

```

#include<iostream>
using namespace std;
class Department {
    char dname[20];
public:
    Department() { setName("None"); }
    Department(char *n) { setName(n); }
    void setName(char* n) { strcpy(dname, n); }
    char* getDname() { return dname; }
    ~Department() { cout<<"delete "<<dname<<endl; }
};
  
```

```

class Faculty {
    char fname[20];
    int num;
    Department *listDepartment;
public:
    Faculty() { setName("None"); num = 0; listDepartment=0; }
    Faculty(char *n, int k, char d[3][20]) {
        setName(n); num = k;
        listDepartment = new Department[num];
        for (int i=0; i<num; i++)
            listDepartment[i].setName(d[i]);
    }
    void setName(char* n) { strcpy(fname, n); }
    char* getFname() { return fname; }
    char *getDepartment(int i) { return listDepartment[i].getDname(); }
    ~Faculty() {
        delete[] listDepartment;
        cout<<"delete "<<fname<<endl;
    }
};
  
```

```

void main() {
    char d[3][20];

    strcpy(d[0], "Computer Science");
    strcpy(d[1], "Mathematics");
    strcpy(d[2], "Chemistry");

    Faculty *sci = new Faculty("Science", 3, d);

    cout<<"-- Department in Faculty --"<<endl;
    for (int i=0; i<3; i++)
        cout<<sci->getDepartment(i)<<endl;

    cout<<"-- delete Faculty --"<<endl;
    delete sci;
}
  
```

ผลลัพธ์โปรแกรม

```

-- Department in Faculty --
Computer Science
Mathematics
Chemistry
-- delete Faculty --
delete Chemistry
delete Computer Science
delete Science
  
```

*** delete listDepartment ***
 delete listDepartment *** delete listDepartment ***
 delete listDepartment *** delete listDepartment ***

เฟรนด์ฟังก์ชัน Friend function

- ฟังก์ชันที่ไม่ใช่สมาชิกของคลาสจะไม่สามารถเข้าถึงข้อมูลใดๆ ที่อยู่ภายในคลาสได้
- คลาสสามารถอนุญาตให้ฟังก์ชันที่ไม่ใช่สมาชิกของคลาสเข้าถึงข้อมูลในคลาสของตนได้โดยการกำหนดให้ฟังก์ชันนั้นเป็นเฟรนด์ฟังก์ชัน
- เฟรนด์ฟังก์ชันสามารถเข้าถึงข้อมูลได้ถึง **private**, **protected** และ **public** ของคลาส
- เฟรนด์ฟังก์ชันจะเป็นฟังก์ชันที่ถูกประกาศอยู่ในคลาสแต่ไม่ใช่สมาชิกของคลาส
- ในการประกาศว่าเป็นเฟรนด์ฟังก์ชัน จะต้องใส่คีย์เวิร์ด **"friend"** เอาไว้ตอนประกาศฟังก์ชันในคลาส

โปรแกรมที่ 5.4

```
#include<iostream>
using namespace std;
class B;
class A {
    int a;
public:
    A() { a=1; }
    friend int add(A, B);
};
class B {
    int b;
public:
    B() { b=2; }
    friend int add(A, B);
};
int add(A x, B y) {
    A z;
    z.a=x.a+y.b;
    return (z.a);
}
```

```
void main() {
    A s;
    B t;
    cout<<add(s, t)<<endl;
}
```

ผลลัพธ์โปรแกรม
3

โปรแกรมที่ 5.5

```
#include<iostream>
using namespace std;
class B;
class A {
    int a;
public:
    A() { a=1; }
    friend int add(A, B);
    void setA(int i) { a = i; }
    int getA() { return a; }
};
class B {
    int b;
public:
    B() { b=2; }
    friend int add(A, B);
};
int add(A x, B y) {
    A z;
    z.setA(x.getA()+y.b);
    return (z.getA());
}
void main() {
    A s;
    B t;
    cout<<add(s, t)<<endl;
}
```

```
int add(A x, B y) {
    A z;
    z.setA(x.getA()+y.b);
    return (z.getA());
}
```

ผลลัพธ์โปรแกรม
3

โปรแกรมที่ 5.6 Friend class

```
#include<iostream>
using namespace std;
class A {
    int a;
public:
    A() { a=1; }
    friend class B;
};
class B {
    int b;
public:
    B() { b=2; }
    int f1(A &x) { return ++x.a; }
    int f2(A &x) { return --x.a; }
    int f3(A x) { return b*x.a; }
};
void main() {
    A s;
    B t;
    cout<<t.f1(s)<<endl;
    cout<<t.f3(s)<<endl;
    cout<<t.f2(s)<<endl;
}
```

ผลลัพธ์โปรแกรม
2
4
1

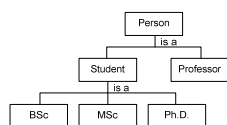
```
class X
{
    int x;
public:
    friend class Y;
};

class Y
{
    int y;
public:
    void f1() { y=2; }
    void f2(X a) {
        cout<<a.x<<endl;
    }
    friend class Z;
};

class Z { /* ... */ }
```

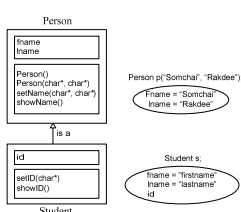
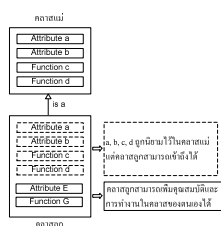
COS2102

บทที่ 6
การรับทอดแบบเดี่ยว
Single Inheritance



– Base class (superclass)
– Derived class (subclass)

- คลาสแม่ หรือ ซุปเปอร์คลาส (superclass) หรือ คลาสพื้นฐาน (base class)
- คลาสลูก หรือ ซับคลาส (subclass) หรือ คลาสสืบต่อ (derived class)
- การรับทอดแอตทริบิวต์และฟังก์ชันมาจากคลาส
- คลาสลูกสืบทอดมรดกมาจากคลาสแม่
- คลาสลูกสามารถเพิ่มแอตทริบิวต์และฟังก์ชันการทำงานอื่นๆ เพิ่มเติมให้กับคลาสของตนเองได้
- เราเรียกความสัมพันธ์ระหว่างคลาสแม่และคลาสลูกแบบนี้ว่าเป็นความสัมพันธ์แบบ “is-a”



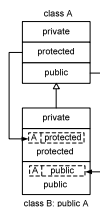
ระดับการเข้าถึงข้อมูล

- ระดับการเข้าถึงข้อมูลของการรับทอดนั้นแบ่งได้เป็น 3 ระดับ
 - public
 - protected
 - private

การรับทอดแบบ public

- การสืบทอดมรดกนั้น ข้อมูลและฟังก์ชันทั้งหมดของคลาสแม่จะสืบทอดลงมายังคลาสลูก ยกเว้น
 - คอนสตรัคเตอร์
 - ดีสทริคเตอร์
 - โอเปอเรเตอร์กำหนดค่า
 - เฟรนดฟังก์ชัน
- คลาสลูกมีชื่อจำกัดในการเข้าถึงข้อมูลและฟังก์ชันที่สืบทอดมา
- ข้อมูลและฟังก์ชันที่อยู่ในส่วน **private** ของคลาสแม่ยังคงรักษาสถานะความเป็นส่วนตัวอยู่ คลาสลูกไม่สามารถเข้าไปแก้ไขได้โดยตรง

การรับทอดแบบ public



```

class A {
public:
    int f1() { return 1; }
    int f2() { return 2; }
};

class B: public A {
public:
    int f1() { return 4; }
    int f3() { return 3; }
    int f4() { return f1(); }
    int f5() { return A::f1(); }
};

void main() {
    A a; B b;

    cout<<a.f1()<<endl;
    cout<<b.f1() <<endl;
    cout<<b.f3() <<endl;
    cout<<b.f2() <<endl;
    cout<<b.f4() <<endl;
    cout<<b.f5() <<endl;
    cout<<b.A::f1() <<endl;
}

1
4
3
2
4
1
1

```

โปรแกรมที่ 6.1

```
#include<iostream>
using namespace std;
class Person {
    char fname[20];
    char lname[20];
public:
    Person() {
        setName("firstname", "lastname");
    }
    Person(char *f, char *l) {
        setName(f, l);
    }
    void setName(char *f, char *l) {
        strcpy(fname, f);
        strcpy(lname, l);
    }
    void showName() {
        cout<<fname<<" "<<lname<<endl;
    }
};
```

```
class Student: public Person {
    char id[10];
public:
    void setID(char *i) { strcpy(id, i); }
    void showID() {
        cout<<"Student ID: "<<id<<endl;
    }
};

void main() {
    Person p("Somchai", "Rakdee");
    p.showName();
    Student s;
    s.setName("Somsamorn", "Meesuk");
    s.setID("12345");
    s.showName();
    s.showID();
}
```

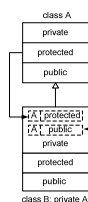
ผลลัพธ์โปรแกรม

```
Somchai Rakdee
firstname lastname
Somsamorn Meesuk
Student ID: 12345
```

การรับทอดแบบ protected



การรับทอดแบบ private



โปรแกรมที่ 6.2

```
#include<iostream>
using namespace std;
class A {
    private:
        int a1;
    protected:
        int a2;
    public:
        int a3;
};
```

```
class B: protected A {
    private:
        int b1;
    protected:
        int b2;
    public:
        int b3;
    void fB() {
        b1 = a1; // error: A::a1 is private
        b2 = a2; // ok
        b3 = a3; // ok
    }
};
```

```
class A {
    private:
        int a1;
    protected:
        int a2;
    public:
        int a3;
};

class B: protected A {
    private:
        int b1;
    protected:
        int b2;
    public:
        int b3;
};

class C: public B {
    private:
        int c1;
    protected:
        int c2;
    public:
        int c3;
    void fC() {
        c1 = a1; //error: A::a1 is private
        c2 = a2; // ok
        c3 = a3; // ok
    }
};
```

class A {	class B: protected A {	class C: public B {
private:	private:	private:
int a1;	int b1;	int c1;
protected:	protected:	protected:
int a2;	int b2;	int c2;
public:	public:	public:
int a3;	int b3;	int c3;


```
void main() {
    A w;
    w.a1=1; // error: A::a1 is private
    w.a2=1; // error: A::a2 is protected
    w.a3=1; // ok

    B v;
    v.a1=1; // error: A::a1 is private
    v.a2=1; // error: A::a2 is protected
    v.a3=1; // error: A::a3 is inaccessible
    v.b1=1; // error: B::b1 is private
    v.b2=1; // error: B::b2 is protected
    v.b3=1; // ok

    C y;
    y.a1=1; // error: A::a1 is private
    y.a2=1; // error: A::a2 is protected
    y.a3=1; // error: A::a3 is inaccessible
    y.b1=1; // error: B::b1 is private
    y.b2=1; // error: B::b2 is protected
    y.b3=1; // ok
}
```

โปรแกรมที่ 6.3

```
#include<iostream>
using namespace std;
class A {
    private:
        int a1;
    protected:
        int a2;
    public:
        int a3;
};
```

```
class B: private A {
    private:
        int b1;
    protected:
        int b2;
    public:
        int b3;
    void fB() {
        b1 = a2; // ok
        b2 = a3; // ok
    }
};
```

```
class A {
    private:
        int a1;
    protected:
        int a2;
    public:
        int a3;
};

class B: private A {
    private:
        int b1;
    protected:
        int b2;
    public:
        int b3;
};

class C: public B {
    private:
        int c1;
    protected:
        int c2;
    public:
        int c3;
    void fC() {
        c1 = a2; // error: A::a2 is protected
        c2 = a3; // error: A::a3 is inaccessible
        c3 = b3; // ok
    }
};
```

```

class A {
private:
    int a1;
protected:
    int a2;
public:
    int a3;

void main() {
    A w;
    w.a3=1; // ok

    B x;
    x.a3=1; // error: A::a3 is inaccessible
    x.b3=1; // ok

    C y;
    y.a3=1; // error: A::a3 is inaccessible
    y.b3=1; // ok
    y.c3=1; // ok
}

```

โปรแกรมที่ 6.4 คอมไพล์และรันโปรแกรมเพื่อตรวจสอบการเข้าถึงสมาชิก

```

#include<iostream>
using namespace std;
class A {
public:
    A() { a=0; cout<<"A"<<endl; }
    A(int i) { a=i; cout<<a<<endl; }
    ~A() { cout<<"bye A"<<endl; }
    void f() {
        a++;
        cout<<"A::f()<<endl;
    }
};

class B: public A {
    int b;
public:
    B() { b=0; cout<<"B"<<endl; }
    B(int i) { b=i; cout<<b<<endl; }
    ~B() { cout<<"bye B"<<endl; }
    void f() {
        b++;
        cout<<"B::f()<<endl;
    }
};

class C: public B {
    int c;
public:
    C() { c=0; cout<<"C"<<endl; }
    C(int i) { c=i; cout<<c<<endl; }
    ~C() { cout<<"bye C"<<endl; }
};

```

```

void main() {
    A x;
    B y;
    C z;
    A m(1);
    B n(2);
    C p(3);
    y.f();
    z.f();
}

```

ผลลัพธ์โปรแกรม

```

A
A
B }   จาร B y;
A }   Bye A
A m(1); }   Bye A
B n(2); }   จาร C z;
C p(3); }   Bye B
1 }   Bye A
A }   จาร A m(1);
2 }   จาร B n(2);
A }   Bye C
B }   Bye B
B }   จาร C p(3);
3 }   Bye A
B::f() }   จาร y.f();
C::f() }   จาร z.f();
bye A }   Bye A

```

โปรแกรมที่ 6.5

```

#include<iostream>
using namespace std;
class A {
public:
    A() { cout<<"A"<<endl; }
    A(int i) { cout<<i<<endl; }
};

class B: public A {
public:
    B() { cout<<"B"<<endl; }
    B(int i, int j):A(i) { cout<<j<<endl; }
};

class C: public B {
public:
    C() { cout<<"C"<<endl; }
    C(int i, int j, int k):B(j) { cout<<k<<endl; }
};

void main() {
    B m(4,5);
    C n;
    C p(6,7,8);
}

```

ผลลัพธ์โปรแกรม

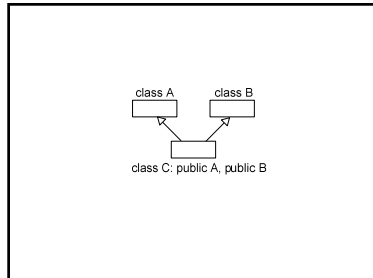
```

4
5
A
B
C
6
7
8

```

COS2102

บทที่ 7
การรับทอดแบบหลายแบบ
Multiple Inheritance



โปรแกรมที่ 7.1

```
#include<iostream>
using namespace std;
class A {
protected:
    int a1;
public:
    A() { a1=a2=0; cout<<"A"<<endl; }
    A(int i) {
        a1=i; a2=++i;
        cout<<"a1="<<a1<<" a2="<<a2<<endl;
    }
    ~A() { cout<<"bye A"<<endl; }
};

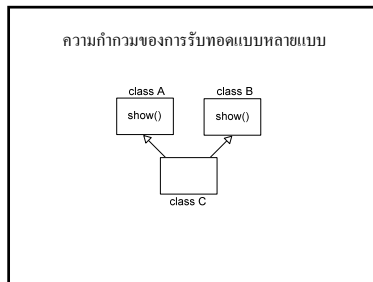
class B {
    int b1;
protected:
    int b2;
public:
    B() { b1=b2=0; cout<<"B"<<endl; }
    B(int i) {
        b1=i; b2=++i;
        cout<<"b1="<<b1<<" b2="<<b2<<endl;
    }
    ~B() { cout<<"bye B"<<endl; }
};
```

ผลลัพธ์โปรแกรม

```
class C: public A, public B {
    int c;
public:
    C() { c=0; cout<<"C"<<endl; }
    C(int i, int j, int k):A(k), B(j) {
        c=i;
        cout<<"c="<<c<<endl;
    }
    ~C() { cout<<"bye C"<<endl; }
    int f() { return (a2+b2); }
};

void main() {
    C m;
    C n(1);
    C p(2,3,4);
    cout<<p.f()<<endl;
}
```

จาก C m; A
 B
 C
จาก C n(1); A
 B
 C
จาก C p(2,3,4); a1=3 a2=4
 b1=4 b2=5
 c=2
 9
 จาก cout<<p.f()<<endl;
 bye C
 bye B
 bye A
 bye C
 bye B
 bye A
 bye C
 bye B
 bye A
 bye C
 bye B
 bye A



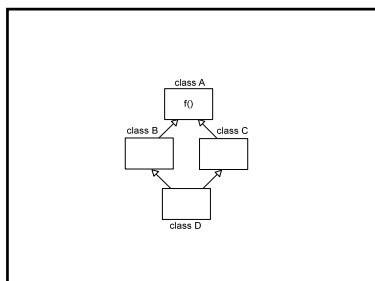
โปรแกรมที่ 7.2

```
#include<iostream>
using namespace std;
class A {
public:
    void show() { cout<<"In class A"<<endl; }
};

class B {
public:
    void show() { cout<<"In class B"<<endl; }
};

void main() {
    C m;
    m.show(); // error: request for member 'show' is ambiguous
    m.A::show(); // ok
    m.B::show(); // ok
}
```

```
class C: public A, public B {
public:
    void f() {
        show(); // error
        A::show(); // ok
        B::show(); // ok
    }
};
```



โปรแกรมที่ 7.3

```
#include<iostream>
using namespace std;
class A {
public:
    void f() { cout<<"f"<<endl; }
};

class B: public A { };
class C: public A { };
class D: public B, public C {
public:
    void show() { f(); // error }
};

void main() {
    D m;
}
```

โปรแกรมที่ 7.4

```
#include<iostream>
using namespace std;
class A {
public:
    A() { cout<<"A"<<endl; }
};

class B: public A {
public:
    B() { cout<<"B"<<endl; }
};

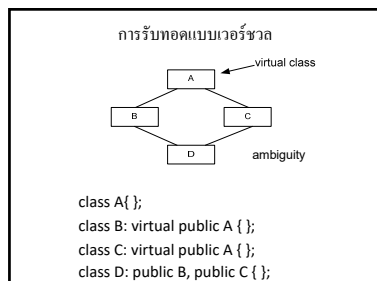
class C: public A {
public:
    C() { cout<<"C"<<endl; }
};

class D: public B, public C {
public:
    D() { cout<<"D"<<endl; }
};

void main() {
    D m;
}
```

ผลลัพธ์โปรแกรม

A
B
C
D



โปรแกรมที่ 7.5

```

#include<iostream>
using namespace std;
class A {
public:
    A() { cout<<"A"<<endl; }
    void f() { cout<<"f()"<<endl; }
};
class B: virtual public A {
public:
    B() { cout<<"B"<<endl; }
};
class C: virtual public A {
public:
    C() { cout<<"C"<<endl; }
};
class D: public B, public C {
public:
    D() { cout<<"D"<<endl; }
    void show() { f(); }
};
void main() {
    D m;
    m.show();
}

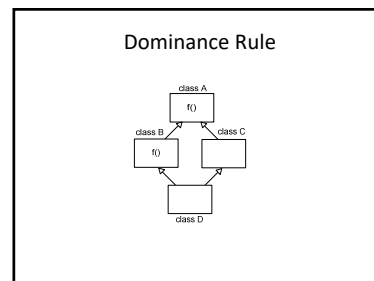
```

ผลลัพธ์โปรแกรม

```

A
B
C
D
f()

```



โปรแกรมที่ 7.6

```

#include<iostream>
using namespace std;
class A {
public:
    A() { cout<<"A"<<endl; }
    void f() { cout<<"f()" in A<<endl; }
};
class B: virtual public A {
public:
    B() { cout<<"B"<<endl; }
    void f() { cout<<"f()" in B<<endl; }
};
class C: virtual public A {
public:
    C() { cout<<"C"<<endl; }
};
class D: public B, public C {
public:
    D() { cout<<"D"<<endl; }
    void show() { f(); }
};
void main() {
    D m;
    m.show();
}

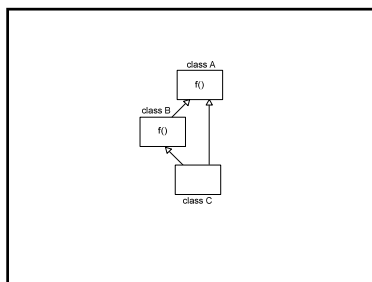
```

ผลลัพธ์โปรแกรม

```

A
B
C
f() in B

```



โปรแกรมที่ 7.7

```

#include<iostream>
using namespace std;
class A {
public:
    A() { cout<<"A"<<endl; }
    void f() { cout<<"f()" in A<<endl; }
};
class B: virtual public A {
public:
    B() { cout<<"B"<<endl; }
    void f() { cout<<"f()" in B<<endl; }
};
class C: public B, virtual public A {
public:
    C() { cout<<"C"<<endl; }
    void show() { f(); }
};
void main() {
    C m;
    m.show();
}

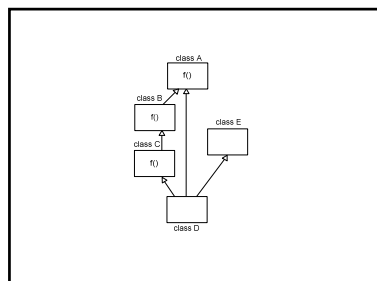
```

ผลลัพธ์โปรแกรม

```

A
B
C
f() in B

```



โปรแกรมที่ 7.8

```

#include<iostream>
using namespace std;
class A {
public:
    A() { cout<<"A"<<endl; }
    void f() { cout<<"f()" in A<<endl; }
};
class B: virtual public A {
public:
    B() { cout<<"B"<<endl; }
    void f() { cout<<"f()" in B<<endl; }
};
class C: public B {
public:
    C() { cout<<"C"<<endl; }
    void f() { cout<<"f()" in C<<endl; }
};
class E {
public:
    E() { cout<<"E"<<endl; }
};
class D: virtual public A, public C, public E {
public:
    D() { cout<<"D"<<endl; }
    void show() { f(); }
};
void main() {
    D m;
    m.show();
}

```

ผลลัพธ์โปรแกรม

```

A
B
C
E
D
f() in C

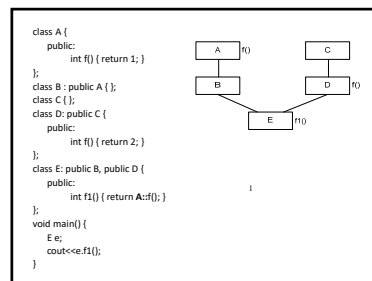
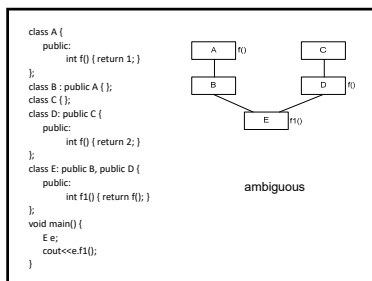
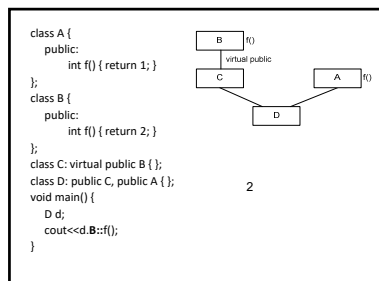
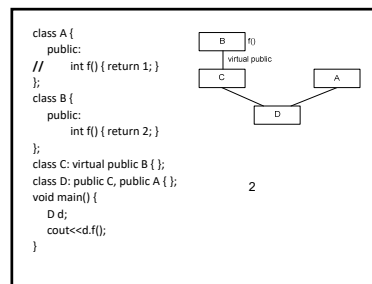
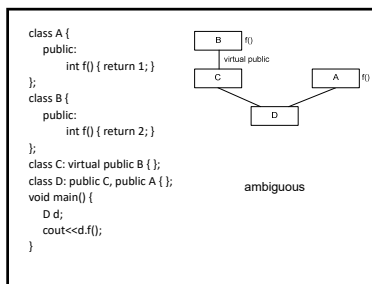
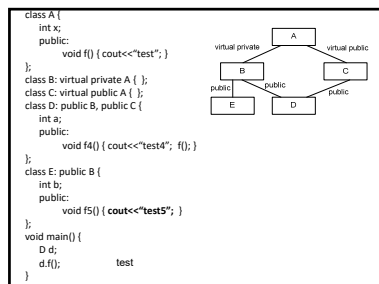
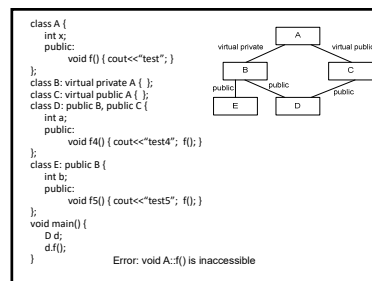
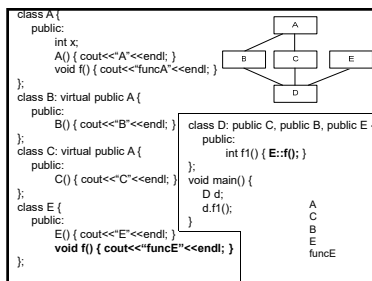
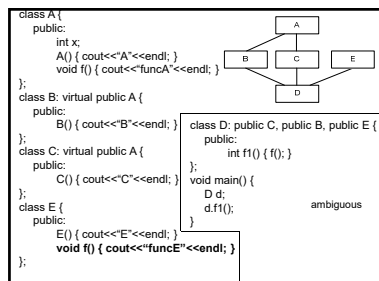
```

โปรแกรมที่ 7.9

```

class A {
public:
    int x;
    A() { cout<<"A"<<endl; }
    void f() { cout<<"funcA"<<endl; }
};
class B: virtual public A {
public:
    B() { cout<<"B"<<endl; }
};
class C: virtual public A {
public:
    C() { cout<<"C"<<endl; }
};
class E {
public:
    E() { cout<<"E"<<endl; }
};
class D: public C, public B, public E {
public:
    int f1() { f(); }
    void main() {
        D d;
        d.f1();
    }
};

```



COS2102

บทที่ 8
โอเพอร์เรเตอร์โอเวอร์โหลดดิ้ง
Operator Overloading

Function Overloading

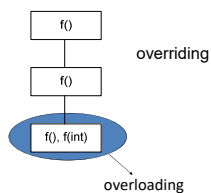
- the same function name to be applied with different types and quantities of arguments.
 - ชื่อฟังก์ชันเหมือนกัน
 - ชนิดของ **parameter** ต่างกัน
 - จำนวน **parameter** ต่างกัน

```
void Display (char *a) { cout<<a<<endl; }
void Display (int a) { cout<<a<<endl; }
void Display (double a) { cout<<a<<endl; }
```

```
void main() {
    Display("Hello");
    Display(123456789);
    Display(3.1415);
    Display(987654321);
    Display(15);
}
```

```
void Display (char *a) { cout<<a<<endl; }
void Display (int a) { cout<<a<<endl; }
void Display (double a) { cout<<a<<endl; }
int Display (int a) { cout<<a<<endl; }
void main() {
    Display("Hello");
    Display(123456789);
    Display(3.1415);
    Display(987654321);
    Display(15);
}
```

error



โปรแกรมที่ 8.1 ฟังก์ชันโอเวอร์โหลดดิ้ง

```
#include <iostream>
using namespace std;
class MyInt {
    int i;
public:
    MyInt() { i=0; }
    void f() { show(); }
    void f(int i) { i=i; show(); }
    void f(int i, int j) { i=i*j; show(); }
    void f(double i) { cout<<"Sorry, I am integer"<<endl; }
    void show() { cout<<i<<endl; }
};

void main() {
    MyInt x;
    x.f();
    x.f(2);
    x.f(2.0);
    x.f(2,3);
}
```

ผลลัพธ์โปรแกรม

```
0
2
Sorry, I am integer
6
```

```
class Foo {
public:
    void func(int a);
private:
    void func(char a);
};
void main() {
    Foo f;
    f.func('a');
}
```

error

```
void f(int x) { x=x+1; }
void f(float y) { y=y-1; }
void main() {
    f('A');
}
```

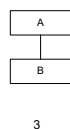
66

```
class A {
public:
    int f(int i) { return i+1; }
};
class B: public A {
public:
    int f(float j) { return j+10; }
};
void main() {
    B b;
    cout << b.f(2);
}
```



12

```
class A {
public:
    int f(int i) { return i+1; }
};
class B: public A {
public:
    int f(float j) { return j+10; }
};
void main() {
    B b;
    cout << b.A::f(2);
}
```



```
a3 = a1.add(a2)
```

หรือ

```
a3.add(a1, a2);
```

ใช้โอเปอเรเตอร์บวก + แทนการใช้ฟังก์ชัน add

```
a3 = a1+a2;
```

เขียนได้ทั้งรูปแบบนี้ก็คือ

```
a3 = a1.operator + (a2);
```

Operator Overloading

&x; → x.operator & ();

x+y; → x.operator + (y); กรณี member

→ operator + (x, y); กรณี friend

- Member of class
- friend

แนวทางในการสร้างโอเปอเรเตอร์โอเวอร์โหลด

- สามารถโอเวอร์โหลดโอเปอเรเตอร์ที่มีอยู่แล้วเท่านั้น ไม่สามารถนิยามโอเปอเรเตอร์ขึ้นมาใหม่ได้
 - เช่น เราไม่สามารถนิยามโอเปอเรเตอร์ ** ขึ้นมาใช้ใน C++ ได้
- ไม่สามารถเปลี่ยนแปลงประเภทของโอเปอเรเตอร์ได้
 - โอเปอเรเตอร์ยูนิวารี (unary operator) ก็ยังคงเป็นยูนิวารี
 - โอเปอเรเตอร์ไบนารี (binary operator) ก็ยังคงเป็นไบนารี
 - กรณีโอเปอเรเตอร์อยู่ 4 ตัวที่ทำหน้าที่ทั้งสองแบบคือเป็นได้ทั้งยูนิวารีและไบนารี โอเปอเรเตอร์ทั้ง 4 ตัวนี้คือ + - * และ &

แนวทางในการสร้างโอเปอเรเตอร์โอเวอร์โหลด

- ลำดับการทำงานของโอเปอเรเตอร์ไม่เปลี่ยนแปลง
 - เช่น โอเปอเรเตอร์ * จะถูกทำก่อนโอเปอเรเตอร์ +
- โอเปอเรเตอร์ที่ไม่สามารถนำมาโอเวอร์โหลดได้มีดังนี้
 - member access operator
 - * dereferencing pointer to member
 - :: scope resolution operator
 - ? : conditional operator
 - # preprocessor convert to string
 - ## preprocessor concatenate

แนวทางในการสร้างโอเปอเรเตอร์โอเวอร์โหลด

- โอเปอเรเตอร์ = () [] และ -> จะต้องถูกโอเวอร์โหลดเป็นสมาชิกของคลาสเท่านั้น ไม่สามารถทำเป็นเฟรเนด์ได้
- การโอเวอร์โหลดโอเปอเรเตอร์ ++ และ -- นั้นควรจะให้โอเวอร์โหลดทั้งกรณีพรีฟิกซ์ (prefix) และโพสต์ฟิกซ์ (postfix)
- การโอเวอร์โหลดโอเปอเรเตอร์ [] นั้นควรจะมีการคืนค่าโดยการอ้างอิง เนื่องจากในบางครั้งโอเปอเรเตอร์ [] อาจจะถูกใช้เพื่อเข้าถึงของเครื่องหมายกำหนดค่า

โอเปอเรเตอร์ยูนิวารี unary operator

โอเปอเรเตอร์ยูนิวารี จะกระทำกับตัวถูกกระทำ (operand) เพียงตัวเดียว

!	&	~	*	+	++	-	--	cast operators
---	---	---	---	---	----	---	----	----------------

โปรแกรมที่ 8.2

```
#include<iostream>
using namespace std;
class Counter {
    int count;
public:
    Counter() { count=0; }
    int getCount() { return count; }
    void operator++ () { ++count; }
    friend void operator--(Counter &x) { x.count--; }
};

void main() {
    Counter a;
    cout<<a.getCount()<<endl;
    ++a;
    ++a;
    cout<<a.getCount()<<endl;
    --a;
    cout<<a.getCount()<<endl;
}
```

ผลลัพธ์โปรแกรม

0

2

1

โปรแกรมที่ 8.3

```
#include<iostream>
using namespace std;
class Counter {
    int count;
public:
    Counter() { count=0; }
    Counter(int c) { count=c; }
    int getCount() { return count; }
    Counter operator++ () { // prefix
        ++count;
        return *this;
    }
    Counter operator++ (int) { // postfix
        return Counter(++count);
    }
    friend Counter operator--(Counter &x) { // prefix
        x.count--;
        return x;
    }
}
```

```

friend Counter operator--(Counter &x, int) { // postfix
    x.count--;
    return x;
}

};

void main() {
    Counter a, b, c, d, e;

    b++;
    c=a++;
    a++;
    cout<<"a"<<a.getCount()<<endl;
    cout<<"b"<<b.getCount()<<endl;
    cout<<"c"<<c.getCount()<<endl;
    d=-c;
    e=a--;
    cout<<"d"<<d.getCount()<<endl;
    cout<<"e"<<e.getCount()<<endl;
}

```

ผลลัพธ์โปรแกรม

a=3
b=1
c=1
d=1
e=2

```

Counter operator++ (int) { // postfix
    return Counter(++count);
}

```

```

Counter operator++ (int) { // postfix
    count++;
    Counter temp;
    temp.count = count;
    return temp;
}

```

โปรแกรมที่ 8.4

```

#include<iostream>
using namespace std;
class MyUnary {
public:
    void operator ! () { cout<<"!!!"<<endl; }
    friend void operator & (MyUnary m) { cout<<"&&&"<<endl; }
    void operator ~ () { cout<<"~"<<endl; }
    friend void operator * (MyUnary m) { cout<<"***"<<endl; }
    void operator + () { cout<<"++"<<endl; }
    friend void operator - (MyUnary m) { cout<<"--"<<endl; }
};

void main() {
    MyUnary m;
    !m;
    &m;
    ~m;
    *m;
    +m;
    -m;
}

```

ผลลัพธ์โปรแกรม

!!!
&&&
~

++
--

โอเปอเรเตอร์แปลงชนิดข้อมูล
cast operators

```

int ivar;
float fvar=10.5;
ivar=fvar;

```

```

ivar = int (fvar);

```

โปรแกรมที่ 8.5 แปลงค่าตัวแปรชนิดข้อมูลในคำสั่งให้เหมาะสมกับชนิดข้อมูลเป้าหมายกับภาษา

```

#include <iostream>
using namespace std;
class Rectangle{
    double width, height;
    double area;
public:
    Rectangle() {
        width=height=area=0.0;
    }
    Rectangle(double d) {
        area=d;
        width =1;
        height=d;
    }
    Rectangle(double w, double h) {
        width=w;
        height=h;
        area=width*height;
    }

    Rectangle operator++() {
        return Rectangle(width+1, height+1);
    }
    operator double() { // convert to double
        return area;
    }
    void show() {
        cout<<"width="<<width;
        cout<<" height="<<height<<endl;
        cout<<"area"<<area<<endl;
    };
}

```

```

void main() {
    Rectangle r1(3.0, 4.0), r2(2.5, 3.5);
    double a, b, c;
    a = double(r1); // convert to double
    cout<<"a"<<a<<endl;
    b = r2; // convert to double
    cout<<"b"<<b<<endl;
    c=r1+r1; // convert result of ++r1 to double
    cout<<"c"<<c<<endl;
    Rectangle r3 = 4.5; // convert to Rectangle using 1-arg constructor
    r3.show();
    r3 = 2.0; // convert to Rectangle using 1-arg constructor
    r3.show();
}

```

ผลลัพธ์โปรแกรม

a=12
b=8.75
c=20
width=1
height=4.5
area=4.5
width=1
height=2
area=2

โปรแกรมที่ 8.6 แปลงค่าตัวแปรชนิดข้อมูลในคำสั่งให้เหมาะสมกับภาษา

```

#include<iostream>
using namespace std;
class Rectangle{
    double width, height;
    double area;
public:
    Rectangle() {
        width=height=area=0.0;
    }
    Rectangle(double w, double h) {
        width=w; height=h;
        area=width*height;
    }
    double getMin() {
        return(width>height ? width : height);
    }
    void show() {
        cout<<"Rectangle: "<<width<<" x "<<height<<" = "<<area<<endl;
    }
};

```

```

class Cube {
    double width;
    double volume;
public:
    Cube() { width=volume=0.0; }
    Cube(double w) {
        width = w;
        volume=w*w*w;
    }
    Cube(Rectangle r) { // convert to cube
        width=r.getMin();
        volume=width*width*width;
    }
    operator Rectangle() { // convert to Rectangle
        return Rectangle(width, width);
    }
    void show() {
        cout<<"Cube: "<<width<<" x "<<width;
        cout<<" x "<<width<<" = "<<volume<<endl;
    }
};

```

```

void main() {
    Rectangle r1(3.0, 4.0), r2;
    Cube c1(5.0), c2; // convert to Rectangle
    r2 = c1; // convert to Cube
    c2 = r1;
    r2.show();
    c2.show();
}

```

ผลลัพธ์โปรแกรม

Rectangle: 5 x 5 = 25
Cube: 3 x 3 x 3 = 27

โอเปอเรเตอร์ไบนารี binary operator

+	-	%	%	&	&&	&n	*	**	+	+=
-	==	=>	=>*	/	/=	<	<<	<<=	<=	=
==	>	>=	>>	>>=	^	^=		=		

โปรแกรมที่ 8.7

```
#include<iostream>
using namespace std;
class Three {
    int a, b, c;
public:
    Three() { a=b=c=0; }
    Three(int i, int j, int k) {
        a=i; b=j; c=k;
    }
    Three operator + (Three m) {
        return Three(a+m.a, b+m.b, c+m.c);
    }
    friend Three operator - (Three n, Three m) {
        return Three(n.a-m.a, n.b-m.b, n.c-m.c);
    }
    void show() {
        cout<<"a"<<"<ca<<" b"<<"<cb<<" c"<<"<<<<endl;
    }
};

void main() {
    Three x(1,2,3), y(4,5,6), z, m;
    z=x+y;
    z.show();
    z=y-x;
    z.show();
    m=x+y+z;
    m.show();
}
```

ผลลัพธ์โปรแกรม

```
a=5 b=7 c=9
a=3 b=3 c=3
a=8 b=10 c=12
```

```
z=x+y;          z=x.operator+(y);

z=y-x;          z=operator-(y, x);

z=x+y+z;        z=x.operator+(y).operator+(z);
```

โปรแกรมที่ 8.8

```
#include<iostream>
using namespace std;
class MyString {
    char s[80];
public:
    MyString() { strcpy(s, ""); }
    MyString(char *x) { strcpy(s, x); }
    bool operator < (MyString);
    bool operator == (MyString);
    void show() { cout<<s<<endl; }
};

bool MyString::operator < (MyString m) {
    return (strlen(s) < strlen(m.s)) ? true : false;
}

bool MyString::operator == (MyString m) {
    return (strcmp(s,m.s)==0) ? true : false;
}
```

ผลลัพธ์โปรแกรม

```
INT2103
COS2102
not equal
x >= y
```

```
void main() {
    MyString x("INT2103"), y("COS2102");
    x.show();
    y.show();
    if (x==y) // x.operator==(y)
        cout<<"equal"<<endl;
    else
        cout<<"not equal"<<endl;
    if (x<y) // x.operator<(y)
        cout<<"x < y"<<endl;
    else
        cout<<"x >= y"<<endl;
}
```

โอเปอเรเตอร์ซับสคริปต์ subscript operator

- การโอเวอร์โหลดโอเปอเรเตอร์ซับสคริปต์นั้นจะต้องมีการคืนค่าโดยการอ้างอิง เนื่องจากซับสคริปต์นั้นสามารถอยู่ได้ทั้งทางซ้ายและทางขวาของเครื่องหมายกำหนดค่า เช่น

```
MyArray a;
a[0] = 5;
a[1] = a[0] + 1;
```

- ดังนั้นถ้าซับสคริปต์อยู่ทางซ้ายของเครื่องหมายกำหนดค่า โอเปอเรเตอร์[] จะต้องทำการคืนค่าโดยการอ้างอิง

```
class Elephant {
    int x, y, z;
public:
    Elephant(int i, int j, int k)
    { x=i; y=j; z=k; }
    int operator [] (int i) {
        switch (i) {
            case 1: return x;
            case 2: return y;
            case 3: return z;
            default: return 0;
        }
    }
    void main() {
        Elephant e(3,4,5);
        cout<<e[1] <<e[2] <<e[3] <<e[4];
    }
}
```

3450

โปรแกรมที่ 8.9

```
#include<iostream>
const int MAX = 3;
using namespace std;
class MyArray {
    int array[MAX];
public:
    MyArray();
    int &operator [] (int n) {
        if (n<0 || n>=MAX) {
            cout<<"The index number is out of bound\n";
            exit(1);
        }
        return array[n];
    }
};

MyArray::MyArray() {
    for (int i=0; i<MAX; i++)
        array[i]=0;
}
```

```
void main() {
    MyArray a, b;
    int i;
    for (i=0; i<3; i++) {
        a[i] = i*2;
        cout<<"a["<<i<<"]="<<a[i]<<endl;
    }
    for (i=0; i<4; i++) {
        b[i] = a[i]*3;
        cout<<"b["<<i<<"]="<<b[i]<<endl;
    }
    cout<<"subscript operator"<<endl;
}
```

ผลลัพธ์โปรแกรม

```
a[0]=0
a[1]=2
a[2]=4
b[0]=0
b[1]=6
b[2]=12
The index number is out of bound
```

โอเปอเรเตอร์การเรียกฟังก์ชัน function call operator

- จะใช้วงเล็บในการเรียก
- การโอเวอร์โหลดโอเปอเรเตอร์นี้จะต้องสร้างเป็นสมาชิกของคลาส
- พารามิเตอร์ของการโอเวอร์โหลดนั้นจะมีจำนวนเท่าใดก็ได้ หรือไม่มีเลยก็ได้

โปรแกรมที่ 8.10

```
#include <iostream>
const int MAX = 100;
using namespace std;

class MyMatrix
{
    int a[MAX][MAX];
public:
    MyMatrix();
    int &operator()(int i, int j);
    int &operator()(int i);
    int &operator()();
};

MyMatrix::MyMatrix() {
    for (int i=0; i<MAX; i++)
        for (int j=0; j<MAX; j++)
            a[i][j]=0;
}
```

```
int & MyMatrix::operator()(int x, int y)
{
    if (x<0 || x>=MAX || y<0 || y>=MAX) {
        cout<<"The index number is out of bound\n";
        exit(1);
    }
    return a[x][y];
}

int & MyMatrix::operator()(int x)
{
    if (x<0 || x>=MAX) {
        cout<<"The index number is out of bound\n";
        exit(1);
    }
    return a[x][x];
}

int & MyMatrix::operator()()
{
    return a[0][0];
}
```

```
void main()
{
    MyMatrix m;
    for (int i=0; i<3; i++) {
        for (int j=0; j<3; j++) {
            m(i,j)=i+j;
            cout<<m(i,j)<<" ";
        }
        cout<<endl;
    }

    cout<<"m()="<<m()<<endl;

    for (int i=0; i<3; i++)
        cout<<"m("<<i<<"="<<m(i)<<endl;
}
```

ผลลัพธ์โปรแกรม

```
0 1 2
1 2 3
2 3 4
m()=0
m(1)=2
m(2)=4
```

COS2102

บทที่ 9
ฟังก์ชันสมาชิกอื่นๆ
Other Member Functions

- โอเปอเรเตอร์กำหนดค่า (assignment operator)
- ก๊อปปี้คอนสตรัคเตอร์ (copy constructor)
- โอเปอเรเตอร์พินค่า (ostream operator)
- โอเปอเรเตอร์รับค่า (istream operator)

โปรแกรมที่ 9.1

```
#include <iostream>
using namespace std;
class Rectangle {
    double width, height;
    double area;
public:
    Rectangle() {
        width=height=area=0.0;
    }
    Rectangle(double w, double h) {
        width=w; height=h;
        area=width*height;
    }
    void show() {
        cout<<width<<" x ";
        cout<<height<<" = "<<area<<endl;
    }
    Rectangle operator + (Rectangle r) {
        return Rectangle(width+r.width, height+r.height);
    }
}
```

```
Rectangle& operator = (const Rectangle &r) {
    if (this==&r) return *this;
    width=r.width;
    height=r.height;
    area=r.area;
    return *this;
}
Rectangle& operator += (const Rectangle &r) {
    width+=r.width;
    height+=r.height;
    area=width*height;
    return *this;
};

void main() {
    Rectangle r1(1.2), r2(3.4), r3(5.6), r4, r5;
    r5=r4=r2;
    cout<<"r5:"; r5.show();
    cout<<"r4:"; r4.show();
    cout<<"....."<<endl;
    r4+=r3+r2;
    cout<<"r4:"; r4.show();
    cout<<"r3:"; r3.show();
}
```

ผลลัพธ์โปรแกรม

```
r5: 3 x 4 = 12
r4: 3 x 4 = 12
.....
r4: 11 x 14 = 154
r3: 8 x 10 = 80
```

```
r5=r4=r2;    ➔    r5=(r4=r2);
               ➔    r5.operator=(r4.operator=(r2));
```

```
r4=r4+r2;    ควจะไม่ได้เท่ากับ    r4+=r2;
```

```
r4+=r3+r2;    ➔    r4.operator+=(r3.operator+=(r2));
```

```
class Pet {
    char *name;
    int sex; // 0 : male, 1 : female
};

class Color {
    int x;
};

class Dog: public Pet {
    char *breed;
    Color c;
};

void main() {
    Dog d1("Peter", 0, "Thai Bangkaew", 3), d2;
    d2=d1;
}
```

ก่อนกำหนดค่า



หลังกำหนดค่า



```
class Pet {
    char *name;
    int sex; // 0 : male, 1 : female
public:
    Pet() { sex=0; name=0; }
    Pet(char *n, int s) { name=new char[strlen(n)+1]; strcpy(name, n); sex=s; }
    ~Pet() { delete[] name; }
    Pet& operator = (const Pet &p) {
        if (this==&p) return *this;
        sex=p.sex;
        delete[] name;
        if (p.name != 0) {
            name=new char[strlen(p.name)+1]; strcpy(name, p.name);
        } else {
            name=0;
        }
        return *this;
    }
    void show() {
        cout<<"Name: "<<name<<endl;
        cout<<"Sex: "<<(sex?" Female":" Male")<<endl;
    }
};
```

โปรแกรมที่ 9.2

```
class Color {
    int x; // 1:black, 2:brown, 3: white
public:
    Color() { x=0; }
    Color(int i) { x=i; }
    char *getColor() {
        switch (x) {
            case 1: return "black";
            case 2: return "brown";
            case 3: return "white";
            default: return "other";
        }
    }
    void show() {
        cout<<"Color: "<<getColor()<<endl;
    }
};
```

```

class Dog: public Pet {
    char *breed;
    Color c;
public:
    Dog() { breed=0; }
    Dog(char *n, int s, char *b, Color j):Pet(n, s)
    { c=j; breed=new char[strlen(b)+1]; strcpy(breed, b); }
    ~Dog() { delete[] breed; }
    Dog& operator = (const Dog &d) {
        if (this==&d) return *this;
        Pet::operator=(d); c=d.c;
        delete[] breed;
        if (d.breed != 0) {
            breed=new char[strlen(d.breed)+1]; strcpy(breed, d.breed);
        }
        else breed=0;
        return *this;
    }
    void show() { Pet::show(); cout<<"Breed: "<<breed<<endl; c.show(); }
};

```

```

void main() {
    Dog d1("Peter", 0, "Thai Bangkaew", 3), d2;
    d2=d1; // d2.operator=(d1);
    d2.show();
}

```

ผลลัพธ์โปรแกรม

Name: Peter
Sex: Male
Breed: Thai Bangkaew
Color: white

```

Dog& operator = (const Dog &d) {
    cout<<"hello my dog"<<endl;
}

```

```

void main()
{
    TextBox t1;
    TextBox t2 = t1;
}

```

~~เขียน default constructor~~
~~เขียน assignment operator~~

ก๊อปปี้คอนสตรัคเตอร์

- Pet p1; เขียนใช้ดีฟอลต์คอนสตรัคเตอร์
 - Pet p2(p1); เขียนใช้ก๊อปปี้คอนสตรัคเตอร์
 - Pet p3=p1; เขียนใช้ก๊อปปี้คอนสตรัคเตอร์
 - P3=p2; เขียนใช้โอเปอเรเตอร์กำหนดค่า
- Pet myPet (Pet p);

โปรแกรมที่ 9.3

```

#include<iostream>
using namespace std;
class Pet {
    char *name;
    int sex; // 0. male, 1. female
public:
    Pet() { sex=0; name=0; }
    ~Pet() { delete[] name; }
    Pet(char *n, int s) {
        name=new char[strlen(n)+1];
        strcpy(name, n); sex=s;
    }
    Pet (const Pet &p) {
        cout<<"copy constructor is called"<<endl;
        sex=p.sex;
        if (p.name != 0) {
            name=new char[strlen(p.name)+1]; strcpy(name, p.name);
        }
        else name=0;
    }
}

```

```

Pet myPet(Pet p) {
    p.show();
    return p;
}
void show() {
    cout<<"Name: "<<name<<endl;
    cout<<"Sex: "<<(sex?" Female":" Male")<<endl;
}
void main() {
    Pet p1("Pukpui", 1);
    Pet p2=p1;
    p2.show();
    p2.myPet(p1);
}

```

ผลลัพธ์โปรแกรม

copy constructor is called
Name: Pukpui
Sex: Female
copy constructor is called
Name: Pukpui
Sex: Female
copy constructor is called

จากตัวอย่างที่ 9.2 ในกรณีที่มีคลาส Dog รับพารามิเตอร์มาจากคลาส Pet สามารถสร้างก๊อปปี้คอนสตรัคเตอร์ของคลาส Dog ได้ดังนี้

```

Dog(const Dog &d) {
    Pet::operator=(d);
    c=d.c;
    if (d.breed != 0) {
        breed=new char[strlen(d.breed)+1];
        strcpy(breed, d.breed);
    }
    else breed=0;
}

```

```

Dog(const Dog &d) :Pet(d), c(d.c) {
    if (d.breed != 0) {
        breed=new char[strlen(d.breed)+1];
        strcpy(breed, d.breed);
    }
    else breed=0;
}

```

```

void vfunc(TextBox t);      Receives object by value
void rfunc(TextBox &t);      Receives object by reference
void pfunc(TextBox *t);      Receives object by pointer
void main() {
    TextBox tbox;
    vfunc(tbox);      เขียน TextBox copy constructor
    rfunc(tbox);      ไม่เขียน TextBox copy constructor
    pfunc(&tbox);      ไม่เขียน TextBox copy constructor
}

```

```
class Foo {
public:
    Foo(Foo &source);
};
void main() {
    Foo f;
}

error    Foo does not have a default constructor
```

โปรแกรมที่ 9.4

```
class Rectangle {
    double width;
    double height;
    double area;
public:
    Rectangle() { width=height=area=0.0; }
    Rectangle(double w, double h) {
        width=w; height=h;
        area=width*height;
    }
    void print(ostream& os) {
        os<<width<<" x "<<height<<" = "<<area<<endl;
    }
    void input(istream& is) {
        cout<<"Enter width: "; is>>width;
        cout<<"Enter height: "; is>>height;
        area=width*height;
    }
};
```

```
ostream& operator<<(ostream& os, Rectangle &r) {
    r.print(os);
    return os;
}
istream& operator>>(istream& is, Rectangle &r) {
    r.input(is);
    return is;
}
void main() {
    Rectangle r1(3.0, 4.0), r2;
    cin>>r2;           // operator>>(cin, r2);
    cout<<r2<<r1;      // operator<<(operator<<(cout, r2), r1);
    system("PAUSE");
}
```

ผลลัพธ์โปรแกรม

```
Enter width: 2.0
Enter height: 3.0
2 x 3 = 6
3 x 4 = 12
```

โปรแกรมที่ 9.5

```
#include<iostream>
using namespace std;
class Rectangle {
    double width, height;
    double area;
public:
    Rectangle() {
        width=height=area=0.0;
    }
    Rectangle(double w, double h) {
        width=w;
        height=h;
        area=width*height;
    }
    friend ostream& operator<<(ostream& os, const Rectangle &r);
    friend istream& operator>>(istream& is, Rectangle &r);
};
```

```
ostream& operator<<(ostream& os, const Rectangle &r) {
    os<<width<<" x "<<height<<" = "<<r.area<<endl;
    return os;
}
istream& operator>>(istream& is, Rectangle &r) {
    cout<<"Enter width: "; is>>r.width;
    cout<<"Enter height: "; is>>r.height;
    r.area=r.width*r.height;
    return is;
}
void main() {
    Rectangle r1(3.0, 4.0), r2;
    cin>>r2;
    cout<<r2<<r1;
}
```

COS2102

บทที่ 10

โพลิมอร์ฟิซึมและเวอร์ชวลฟังก์ชัน
Polymorphism and Virtual Function

```

class Musician {
public:
    void greet();
    virtual void play();
};
void Musician::greet()
{ cout<<"hello\n"; }
void Musician::play()
{ cout<<"play: la la\n"; }

class Trumpeter: public Musician {
public:
    void greet();
    void play();
};
void Trumpeter::greet()
{ cout<<"ho ho\n"; }
void Trumpeter::play()
{ cout<<"play: toot toot\n"; }

void main() {
    Trumpeter t;
    Musician m, *pm;
    m.greet();      hello
    m.play();       play: la la
    t.greet();      ho ho
    t.play();       play: toot toot
    pm=&m;
    pm->greet();    hello
    pm->play();     play: la la
    pm=&t;
    pm->greet();    hello
    pm->play();     play: toot toot
}

```

```

class Musician {
public:
    void greet();
    virtual void play();
};
void Musician::greet()
{ cout<<"hello\n"; }
void Musician::play()
{ cout<<"play: la la\n"; }

class Trumpeter: public Musician {
public:
    void greet();
    void play();
};
void Trumpeter::greet()
{ cout<<"ho ho\n"; }
void Trumpeter::play()
{ cout<<"play: toot toot\n"; }

void main() {
    Musician *pm;
    pm = new Musician;
    pm->greet();    hello
    pm->play();     play: la la
    delete pm;
    pm = new Trumpeter;
    pm->greet();    hello
    pm->play();     play: toot toot
    delete pm;
}

```

```

class Musician {
public:
    void greet();
    virtual void play();
};
void Musician::greet()
{ cout<<"hello\n"; }
void Musician::play()
{ cout<<"play: la la\n"; }

class Trumpeter: public Musician {
public:
    void greet();
    void play();
};
void Trumpeter::greet()
{ cout<<"ho ho\n"; }
void Trumpeter::play()
{ cout<<"play: toot toot\n"; }

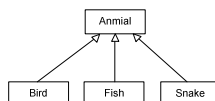
void func(Musician &m)
{ m.play(); }

void main() {
    Musician m;
    Trumpeter t;
    func(m);      play: la la
    func(t);      play: toot toot
}

```

Virtual Function

- สร้างฟังก์ชัน **move** เพื่อแสดงพฤติกรรมรวมการเคลื่อนที่ของสัตว์แต่ละชนิด เช่น
 - นกเคลื่อนที่ด้วยการบิน
 - ปลาจะใช้วิธีการว่ายน้ำ
 - งูจะใช้วิธีการเลื้อย



Animal *p[3]; // array of Bird, Fish and Snake

โดยที่ p[0] เป็นพอยเตอร์ที่ชี้ไปยัง Bird
 p[1] เป็นพอยเตอร์ที่ชี้ไปยัง Fish
 p[2] เป็นพอยเตอร์ที่ชี้ไปยัง Snake

```

for (int i=0; i<3; i++)
    p[i]->move();

```

โปรแกรมที่ 10.1

```

#include<iostream>
using namespace std;
class Animal {
public:
    void show() { cout<<"Animal"<<endl; }
    virtual void move() {
        cout<<"I am moving"<<endl;
    }
};
class Bird: public Animal {
public:
    void show() { cout<<"Bird"<<endl; }
    void move() {
        cout<<"I am flying"<<endl;
    }
};
class Fish: public Animal {
public:
    void show() { cout<<"Fish"<<endl; }
    void move() {
        cout<<"I am swimming"<<endl;
    }
};
class Snake: public Animal {
public:
    void show() { cout<<"Snake"<<endl; }
    void move() {
        cout<<"I am crawling"<<endl;
    }
};

```

```

void main() {
    Animal *p[3];
    Bird b;
    Fish f;
    Snake s;
    p[0] = &b;
    p[1] = &f;
    p[2] = &s;
    for (int i=0; i<3; i++) {
        p[i]->show();
        p[i]->move();
    }
}

```

ไดนามิกโพลิมอร์ฟิซึม

```

void func(Animal &a) {
    a.move();
}

void main() {
    Animal a;
    Bird b;
    Fish f;
    Snake s;
    func(a);
    func(b);
    func(f);
    func(s);
}

```

สแตติกโพลิมอร์ฟิซึม

I am moving
 I am flying
 I am swimming
 I am crawling

```
void func(Animal *a) {
    a->move();
}
```

```
void main() {
    Animal a;
    Bird b;
    Fish f;
    Snake s;

    func(&a);
    func(&b);
    func(&f);
    func(&s);
}
```

ผลลัพธ์โปรแกรม

```
I am moving
I am flying
I am swimming
I am crawling
```

Virtual Destructor

- เราจะเรียกกลไกที่ประกอบไปด้วยเวอร์ชวลฟังก์ชันอย่างน้อย 1 ฟังก์ชันว่า โพลิมอร์ฟิกคลาส (polymorphic class)
- ถ้าคลาสใดเป็นโพลิมอร์ฟิกคลาส คลาสนั้นจะต้องมีเวอร์ชวลดีสตรัคเตอร์ (virtual destructor) ด้วย

โปรแกรมที่ 10.2

```
#include<iostream>
using namespace std;
class Animal {
public:
    Animal() { cout<<"Animal"<<endl; }
    virtual void move() {
        cout<<"I am moving"<<endl;
    }
    virtual ~Animal() {
        cout<<"bye Animal"<<endl;
    }
};
class Bird: public Animal {
public:
    Bird() { cout<<"Bird"<<endl; }
    void move() {
        cout<<"I am flying"<<endl;
    }
    ~Bird() { cout<<"bye Bird"<<endl; }
};
```

```
class Fish: public Animal {
public:
    Fish() { cout<<"Fish"<<endl; }
    void move() {
        cout<<"I am swimming"<<endl;
    }
    ~Fish() { cout<<"bye Fish"<<endl; }
};
class Snake: public Animal {
public:
    Snake() { cout<<"Snake"<<endl; }
    void move() {
        cout<<"I am crawling"<<endl;
    }
    ~Snake() {
        cout<<"bye Snake"<<endl;
    }
};
```

```
void main() {
    Animal *p[3];
    p[0] = new Bird;
    p[1] = new Fish;
    p[2] = new Snake;
    for (int i=0; i<3; i++) {
        p[i]->move();
    }
    for (int i=0; i<3; i++) {
        delete p[i];
    }
}
```

ผลลัพธ์โปรแกรม

```
Animal
Bird
Animal
Fish
Animal
Snake
I am flying
I am swimming
I am crawling
Bye Bird
Bye Animal
Bye Fish
Bye Animal
Bye Snake
Bye Animal
```

```
class Animal {
public:
    ...
    ~Animal() {
        cout<<"bye Animal"<<endl;
    }
};
```

ผลลัพธ์โปรแกรม

```
Animal
Bird
Animal
Fish
Animal
Snake
I am flying
I am swimming
I am crawling
Bye Animal
Bye Animal
Bye Animal
```

โปรแกรมที่ 10.3

```
#include<iostream>
using namespace std;
class Animal {
public:
    virtual void print(ostream &os) { os<<"Animal"<<endl; }
};
class Bird: public Animal {
public:
    void print(ostream &os) { cout<<"Bird"<<endl; }
};
class Fish: public Animal {
public:
    void print(ostream &os) { cout<<"Fish"<<endl; }
};
class Snake: public Animal {
public:
    void print(ostream &os) { cout<<"Snake"<<endl; }
};
```

```
ostream& operator<<(ostream& os, Animal &a) {
    a.print(os);
    return os;
}
```

```
void main() {
    Animal a;
    Bird b;
    Fish f;
    Snake s;

    cout<<a;
    cout<<b;
    cout<<f;
    cout<<s;
}
```

ผลลัพธ์โปรแกรม

```
Animal
Bird
Fish
Snake
```

Abstract Class

```
class Musician {
public:
    virtual void greet(); // virtual function
    virtual void play() = 0; // pure virtual function
};
void main() {
    Musician m;
    Musician *pm;
    pm=new Musician;
}
```

error

ok

error

class Musician {

public:

virtual void play() = 0;

Abstract class

};

class BrassMusician: public Musician {};

Abstract class

class WoodwindMusician: public Musician {

Abstract class

public:

void play()=0;

};

class Trumpeter: public BrassMusician {

Concrete class

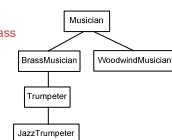
public:

void play();

};

class JazzTrumpeter: public Trumpeter {};

Concrete class



```
#include<iostream>
using namespace std;
class Animal {
public:
    virtual void show()=0; // pure virtual function
};
class Bird: public Animal {
public:
    void show() { cout<<"I am a bird"<<endl; }
};
class Chicken: public Bird {
public:
    void show() { cout<<"I am a chicken"<<endl; }
};
class Hen: public Chicken {
public:
    void show() { cout<<"I am a hen"<<endl; }
};
void func(Animal *a) {
    a->show();
}
```

โปรแกรมที่ 10.4

```
void main() {
    Animal *p[3];
    Bird b;
    Chicken c;
    Hen h;
    p[0] = &b;
    p[1] = &c;
    p[2] = &h;
    for (int i=0; i<3; i++) {
        func(p[i]);
    }

    Animal a; ?
    Animal *a = new Animal; ?
}
```

ผลลัพธ์โปรแกรม

I am a bird
I am a chicken
I am a hen

COS2102

วันที่ 11

พฤษภาคม

Template

```
int square(int x) {
    return (x*x);
}

double square(double x) {
    return (x*x);
}
```

โปรแกรมที่ 11.1 ฟังก์ชันเทมเพลต

```
#include <iostream>
using namespace std;

template <class T>
T square(T x) {
    return (x*x);
}

void main() {
    cout<<square(2)<<endl;
    cout<<square(2.5)<<endl;
}
```

ผลลัพธ์โปรแกรม

```
4
6.25
```

```
#include <iostream>
using namespace std;
class A {
public:
    A(int i, int j) { a=i; b=j; }
    A operator*(A t) { return A(a*t.a, b*t.b); }
    friend ostream& operator<<(ostream& os, const A &r);
};

ostream& operator<<(ostream& os, const A &r) {
    cout<<"["<<r.a<<","<<r.b<<"]"<<endl;
    return os;
}

template <class T>
T square(T x) {
    return (x*x);
}

void main() {
    A y(2,3);
    cout<<square(y)<<endl;
}
```

โปรแกรมที่ 11.2

ผลลัพธ์โปรแกรม

(4,9)

โปรแกรมที่ 11.3

```
#include <iostream>
using namespace std;
class A {
public:
    A(int i, int j) { a=i; b=j; }
    bool operator==(A t) {
        return ((a==t.a && b==t.b)? true : false);
    }
};

template <class T>
int find(T *x, T data, int size) {
    for (int i=0; i<size; i++) {
        if (x[i]==data)
            return i;
        return -1;
    }
}

void main() {
    int x[]={1, 2, 3, 4};
    int x1=2;
    int x2=5;

    float y[]={5.0, 6.0, 7.0, 8.0};
    float y1=7.0;
    float y2=9.0;

    A z[]={A(1,2), A(2,3), A(3,4), A(4,5)};
    A z1(4,5);
    A z2(2,2);

    cout<<"x1 index"<<find(x, x1, 4)<<endl;
    cout<<"x2 index"<<find(x, x2, 4)<<endl;
    cout<<"y1 index"<<find(y, y1, 4)<<endl;
    cout<<"y2 index"<<find(y, y2, 4)<<endl;
    cout<<"z1 index"<<find(z, z1, 4)<<endl;
    cout<<"z2 index"<<find(z, z2, 4)<<endl;
}
```

ผลลัพธ์โปรแกรม

```
x1 index=1
x2 index=-1
y1 index=2
y2 index=-1
z1 index=3
z2 index=-1
```

โปรแกรมที่ 11.4

```
#include <iostream>
using namespace std;
class A {
public:
    A(int i, int j) { a=i; b=j; }
    bool operator==(A t) {
        return ((a==t.a && b==t.b)? true : false);
    }
};

friend bool operator==(A t, int k);
friend bool operator==(int k, A t);
friend bool operator==(A t, float k);
friend bool operator==(float k, A t);

bool operator==(A t, int k) {
    return ((k==t.a && k==t.b)? true : false);
}

bool operator==(int k, A t) {
    return ((k==t.a && k==t.b)? true : false);
}

bool operator==(A t, float k) {
    return false;
}

bool operator==(float k, A t) {
    return false;
}

template <class T, class S>
int find(T *x, S data, int size) {
    for (int i=0; i<size; i++) {
        if (x[i]==data)
            return i;
        return -1;
    }
}
```

```
void main() {
    int x[]={3, 4, 5, 6, 7};
    int x1=6;
    float y[]={3.0, 5.0, 6.0, 7.0, 8.0};
    float y1=4.0;

    A z[]={A(2,2), A(2,3), A(3,3), A(6,6)};
    A z1(3,3);

    cout<<"x1 index"<<find(x, y1, 4)<<endl;
    cout<<"y1 index"<<find(y, x1, 4)<<endl;

    cout<<"x1 index"<<find(x, z1, 4)<<endl;
    cout<<"z1 index"<<find(z, x1, 4)<<endl;

    cout<<"y1 index"<<find(y, z1, 4)<<endl;
    cout<<"z1 index"<<find(z, y1, 4)<<endl;

    cout<<"z1 index"<<find(z, z1, 4)<<endl;
}
```

ผลลัพธ์โปรแกรม

```
x1 index=1
y1 index=2
z1 index=0
z1 index=3
y1 index=-1
z1 index=-1
z1 index=2
```

โปรแกรมที่ 11.5 การหาขอบเขต

```
#include <iostream>
using namespace std;
const int MAX = 80;

template <class T>
class Stack {
public:
    T s[MAX];
    int top;
    Stack() { top = -1; }
    void push(T data) {
        s[++top]=data;
    }
    T pop() {
        return s[top--];
    }
};

void main() {
    Stack<int> s1;
    for (int i=0; i<3; i++)
        s1.push(i);
    for (int i=0; i<3; i++)
        cout<<s1.pop()<<endl;

    Stack<double> s2;
    for (int i=0; i<3; i++)
        s2.push(i*2.0);
    for (int i=0; i<3; i++)
        cout<<s2.pop()<<endl;
}
```

ผลลัพธ์โปรแกรม

```
2
1
0
4
2
0
```

```
template <class T>
class Stack {
    T s[MAX];
    int top;
public:
    Stack();
    void push(T data);
    T pop();
};

template <class T>
Stack<T>::Stack() {
    top = -1;
}

template <class T>
void Stack<T>::push(T data) {
    s[++top]=data;
}

template <class T>
T Stack<T>::pop() {
    return s[top--];
}
```

โปรแกรมที่ 11.6

```
#include<iostream>
using namespace std;
//-----
// Employee
//-----
class Employee {
    char name[80];
public:
    friend ostream& operator<<(ostream& os, Employee& e);
    friend ostream& operator<<(ostream& os, Employee& e);
};
ostream& operator<<(ostream& os, Employee& e) {
    cout<<"Enter name:"; is->e.name;
    return is;
}
ostream& operator<<(ostream& os, Employee& e) {
    os<<e.name;
    return os;
}
```

```
// Link List (class template)
//-----
template<class T>
class Node {
    T data;
    Node *next;
public:
    Node() { next=0; }
    void set(T value, Node *f) {
        data=value; next=f; }
    T &getData() { return data; }
    Node* getNext() { return next; }
};
template<class T>
class LinkList {
    Node<T> *first;
public:
    LinkList() { first=0; }
    ~LinkList() {}
    void add(T value);
    void print(ostream &os);
};
template<class T>
LinkList<T>::~LinkList() {
    Node<T> *temp;
    for (Node<T> *p = first; p; ) {
        temp=p;
        p=p->getNext();
        delete temp;
    }
}
template<class T>
void LinkList<T>::add(T value) {
    Node<T> *n = new Node<T>;
    n->set(value, first);
    first = n;
}
template<class T>
void LinkList<T>::print(ostream &os) {
    for (Node<T> *p = first; p; p=p->getNext())
        os<<p->getData()<<"<endl>";
}
```

```
template<class T>
ostream& operator<<(ostream& os, LinkList<T> &r) {
    r.print(os);
    return os;
}
void main() {
    LinkList<double> myDouble;
    myDouble.add(1.2);
    myDouble.add(2.3);
    myDouble.add(3.4);
    cout<<myDouble;

    LinkList<Employee> myEmp;
    Employee emp;
    for (int i=0; i<3; i++) {
        cin>>emp;
        myEmp.add(emp);
    }
    cout<<myEmp;
}
```

ผลลัพธ์โปรแกรม

```
3.4
2.3
1.2
Enter name:Somkid
Enter name:Pawalai
Enter name:Somchai
Somchai
Pawalai
Somkid
```

โปรแกรมที่ 11.7 การส่งผ่านคลาสแบบพหุไปยังเทมเพลตพหุอาร์กิวเมนต์

```
#include<iostream>
using namespace std;
template<class T>
class Vector {
    T *dataVector;
    int sizeVector;
public:
    Vector() {
        dataVector = 0; sizeVector = 0;
    }
    Vector(int i) {
        sizeVector = i;
        dataVector = new T[sizeVector];
    }
    ~Vector() { delete[] dataVector; }
    T& operator[](int i) {
        return dataVector[i];
    }
    Vector<T>& operator=(Vector<T> &v) {
        if (sizeVector != v.sizeVector) {
            sizeVector = v.sizeVector;
            delete[] dataVector;
            dataVector = new T[sizeVector];
        }
        for (int i=0; i<sizeVector; i++)
            dataVector[i]=v.dataVector[i];
        return *this;
    }
    int getSize() {
        return sizeVector;
    }
};
```

```
template<class T>
class Matrix {
    Vector<Vector<T>> row;
public:
    Matrix(int r, int c) : row(r) {
        for (int i=0; i<r; i++) {
            row[i] = new Vector<T>(c);
        }
    }
    ~Matrix() {
        for (int i=0; i<row.getSize(); i++)
            delete row[i];
    }
    Vector<T>& operator[](int i) {
        return *row[i];
    }
    int getRow() {
        return row.getSize();
    }
    int getColumn() {
        return row[0]->getSize();
    }
    void print(ostream &os) {
        for (int i=0; i<getRow(); i++) {
            for (int j=0; j<getColumn(); j++)
                os<<row[i][j]<<" ";
            cout<<"<endl>";
        }
    }
};
```

row[] จะหมายถึงเวกเตอร์ที่มีสมาชิกเป็นเวกเตอร์ (row[i] จะหมายถึงสมาชิกในเวกเตอร์ row[i])

```
template<class T>
ostream& operator<<(ostream& os, Matrix<T>& m) {
    m.print(os);
    return os;
}
void main() {
    Matrix<int> a(3,4);
    for (int i=0; i<3; i++)
        for (int j=0; j<4; j++)
            a[i][j]=i+k++;
    cout<<"<endl>";

    Matrix<char> b(3,4);
    char k='a';
    for (int i=0; i<3; i++)
        for (int j=0; j<4; j++)
            b[i][j]=k++;
    cout<<"<endl>";
}
```

ผลลัพธ์โปรแกรม

```
0 1 2 3
4 5 6 7
8 9 10 11

a b c d
e f g h
i j k l
```

เทมเพลตและการรับทอด

- กรณีคลาสแม่ไม่ใช่คลาสเทมเพลต แต่คลาสลูกเป็นคลาสเทมเพลต
- กรณีคลาสแม่เป็นคลาสเทมเพลต แต่คลาสลูกไม่ใช่คลาสเทมเพลต
- กรณีคลาสแม่เป็นคลาสเทมเพลต และคลาสลูกเป็นคลาสเทมเพลต

โปรแกรมที่ 11.8 กรณีคลาสแม่ไม่ใช่คลาสเทมเพลต แต่คลาสลูกเป็นคลาสเทมเพลต

```
#include<iostream>
using namespace std;
class A {
    int a;
public:
    void setA(int i) { a=i; }
    int getA() { return a; }
};
template<class T>
class B : public A {
    T b;
public:
    void setB(int i, T k) {
        b=k; setA(i);
    }
    void showB() {
        cout<<getA()<<"<endl>";
        cout<<b<<"<endl>";
    }
};
```

เทมเพลตพหุอาร์กิวเมนต์จะสามารถใช้ได้เฉพาะในคลาส B ซึ่งเป็นคลาสลูกเท่านั้น
คลาส A จะไม่รู้จักเทมเพลตพหุอาร์กิวเมนต์ของคลาส B

```
class Employee {
    char name[80];
public:
    Employee() {
        set("");
    }
    Employee(char *n) {
        set(n);
    }
    void set(char *n) {
        strcpy(name,n);
    }
    friend ostream& operator<<(ostream& os, Employee& e);
};
ostream& operator<<(ostream& os, Employee& e) {
    os<<e.name;
    return os;
}
```

```
void main() {
    B<double> y;
    y.setB(2, 8.8);
    y.showB();

    B<Employee> e;
    e.setB(3, Employee("Somkid"));
    e.showB();
    e.setA(5);
    e.showB();
}
```

ผลลัพธ์โปรแกรม

```
2
8.8
Somkid
5
Somkid
```

โปรแกรมที่ 11.9 กรณีคลาสแม่เป็นคลาสธรรมดา แต่คลาสลูกเป็นคลาสธรรมดา

```
#include<iostream>
using namespace std;

class Employee {
    char name[80];
public:
    Employee() { set(""); }
    Employee(char *n) { set(n); }
    void set(char *n) { strcpy(name,n); }
    friend ostream& operator<<(ostream& os, Employee& e);
};

ostream& operator<<(ostream& os, Employee& e) {
    os<<e.name;
    return os;
}
```

คลาสลูกสามารถใช้ชื่อสมาชิกของคลาสแม่ได้โดยการระบุอย่างชัดเจนว่า
คลาสลูกนี้ต้องการใช้ชื่อสมาชิกของคลาสแม่ที่เป็นชื่อของข้อมูลชนิดใด

```
template <class T>
class A {
    T a;
public:
    void setA(T i) { a=i; }
    T& getA() { return a; }
};

class B : public A<Employee> {
    int b;
public:
    void setB(Employee j, int k) {
        b=k;
        setA(j);
    }
    void showB() {
        cout<< getA() <<endl;
        cout<<b<<endl;
    }
};

void main() {
    B y;
    Employee emp1;
    emp1.set("Somkid");

    y.setB(emp1, 4);
    y.showB();

    y.setA(Employee("Malee"));
    y.showB();
}
```

ผลลัพธ์โปรแกรม

```
Somkid
4
Malee
4
```

โปรแกรมที่ 11.10 กรณีคลาสแม่เป็นคลาสธรรมดา และคลาสลูกเป็นคลาสธรรมดา

```
#include<iostream>
using namespace std;

class Employee {
    char name[80];
public:
    Employee() { set(""); }
    Employee(char *n) { set(n); }
    void set(char *n) { strcpy(name,n); }
    friend ostream& operator<<(ostream& os, Employee& e);
};

ostream& operator<<(ostream& os, Employee& e) {
    os<<e.name;
    return os;
}
```

เทมเพลตพารามิเตอร์ของคลาสลูกสามารถใช้เป็นเทมเพลตพารามิเตอร์ของคลาสแม่ได้

```
template <class T>
class A {
    T a;
public:
    void setA(T i) { a=i; }
    T& getA() { return a; }
};

template <class S, class R>
class B : public A<S> {
    R b;
public:
    void setB(S j, R k) {
        b=k;
        setA(j);
    }
    void showB() {
        cout<< A<S>::getA() <<endl;
        cout<<b<<endl;
    }
};

void main() {
    B<int, char*> y;
    y.setB(3, "hello");
    y.showB();
    y.setA(5);
    y.showB();

    cout<<"....."<<endl;
    B<Employee, Employee> e;
    e.setB(Employee("Somkid"), Employee("Malee"));
    e.showB();
    e.setA(Employee("Bird"));
    e.showB();
}
```

ผลลัพธ์โปรแกรม

```
3
hello
5
hello
.....
Somkid
Malee
Bird
Malee
```

COS2102

บทที่ 12
การจัดการความผิดปกติ
Exception Handling

โปรแกรมที่ 12.1 *คือวิธี try catch and throw*

```
#include <iostream>
using namespace std;
void main() {
    try {
        throw 5;
    }
    catch (int e) {
        cout<<"Exception: "<<e <<endl;
    }
    cout<<"Hello World"<<endl;
}
```

ผลลัพธ์โปรแกรม

Exception: 5
Hello World

โปรแกรมที่ 12.2

```
void main() {
    try {
        char* s = "hello";
        throw s;
        cout<<"<<endl;
    }
    catch (double d) {
        cout<<"Exception: "<<d<<endl;
    }
    catch (int i) {
        cout<<"Exception: "<<i<<endl;
    }
    catch (char c) {
        cout<<"Exception: "<<c<<endl;
    }
    catch (...) {
        cout << "Other exceptions"<<endl;
    }
    cout<<"Hello World"<<endl;
}
```

ผลลัพธ์โปรแกรม

Other exceptions
Hello World

โปรแกรมที่ 12.3

```
#include <iostream>
using namespace std;
class A {
public:
    A() { cout << "A"<<endl; }
    ~A() { cout << "~A"<<endl; }
};

class B {
public:
    B() { cout << "B"<<endl; }
    ~B() { cout << "~B"<<endl; }
};

void main() {
    try {
        cout << "Hello A and B" << endl;
        A a;
        B b;
        throw(1);
        B bb;
        cout<<"I am ok"<<endl;
    }
    catch (int) {
        cout<<"catch int" << endl;
    }
    catch (...) {
        cout << "Umm..." << endl;
    }
    cout << "Happy" << endl;
}
```

ผลลัพธ์โปรแกรม

Hello A and B
A
B
~B
~A
catch int
Happy

โปรแกรมที่ 12.4

```
#include <iostream>
using namespace std;
void main() {
    try {
        try {
            char* s = "hello";
            throw s;
            cout<<"<<endl;
        }
        catch (char* s) {
            throw;
        }
    }
    catch (double d) {
        cout<<"Exception: "<<d<<endl;
    }
    catch (char* s) {
        cout << "Exception: "<<s<<endl;
    }
    cout<<"Hello World"<<endl;
}
```

ผลลัพธ์โปรแกรม

Exception: hello
Hello World

ฟังก์ชันและการดักจับความผิดปกติ

- float func1(char c) throw(int);
- char* func2(int j) throw(int, float);
- int func3(float a) throw(); ไม่อนุญาตให้โยนชนิดของ
ความผิดปกติใดๆ ออกจากฟังก์ชันเลย
- int func4(int i); อนุญาตให้โยนความผิดปกติทุกชนิดออกจาก
ฟังก์ชันได้

int func5(int k) noexcept(false); //โยนออกมาได้ทุกอย่าง

ลำดับการ error มีความสำคัญ

โปรแกรมที่ 12.5

```
#include <iostream>
using namespace std;
double divide(double x, double y)
throw(int) {
    if (y==0)
        throw 0;
    return x/y;
}

void main() {
    try {
        cout<<divide(10, 2)<<endl;
        cout<<divide(10, 0)<<endl;
        cout<<divide(10, 5)<<endl;
    }
    catch(int i) {
        cout<<"divide by zero"<<endl;
    }
    cout<<"Hello World"<<endl;
}
```

ผลลัพธ์โปรแกรม

5
divide by zero
Hello World

โปรแกรมที่ 12.6

```
#include <iostream>
using namespace std;
class A {
public:
    A() { cout<<"A"<<endl; }
};

class B: public A {
public:
    B() { cout<<"B"<<endl; }
};

class C: public B {
public:
    C() { cout<<"C"<<endl; }
};

void f() throw (A) {
    B b;
    throw b;
}

void main() {
    try {
        f();
        cout<<"in try block"<<endl;
    }
    catch (C) {
        cout<<"catch C"<<endl;
    }
    catch (B) {
        cout<<"catch B"<<endl;
    }
    catch (A) {
        cout<<"catch A"<<endl;
    }
    catch (...) {
        cout<<"no catch"<<endl;
    }
    cout<<"Hello World"<<endl;
}
```

ผลลัพธ์โปรแกรม

A
B
catch B
Hello World

ให้โยนได้ว่าเป็นตระกูลของ A

```
void main() {
    try {
        f();
        cout<<"in try block"<<endl;
    }
    catch (A) {
        cout<<"catch A"<<endl;
    }
    catch (B) {
        cout<<"catch B"<<endl;
    }
    catch (C) {
        cout<<"catch C"<<endl;
    }
    catch (...) {
        cout<<"no catch"<<endl;
    }
    cout<<"Hello World"<<endl;
}
```

ผลลัพธ์โปรแกรม

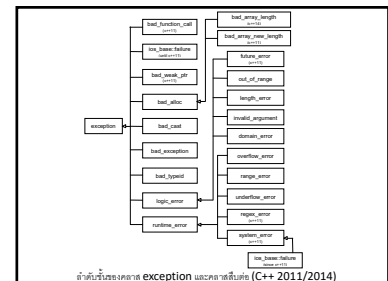
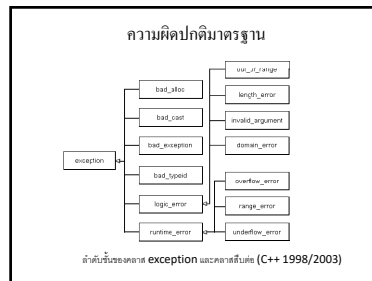
A
B
catch A
Hello World

โปรแกรมที่ 12.7

```
#include <iostream>
using namespace std;
class A {
public:
    A() { cout<<"A"<<endl; }
    virtual ~A() { cout<<"~A"<<endl; }
    virtual void f() throw (int, double) {
        throw 3.0;
    }
};
class B: public A {
public:
    B() { cout<<"B"<<endl; }
    ~B() { cout<<"~B"<<endl; }
    void f() throw(int) {
        throw 0;
    }
};

void main() {
    A *x = new B();
    try {
        x->f();
        cout<<"hello x"<<endl;
    }
    catch (int i) {
        cout<<"Exception: "<<i<<endl;
    }
    catch (double d) {
        cout<<"Exception: "<<d<<endl;
    }
    catch (...) {
        cout<<"Other exceptions"<<endl;
    }
    delete x;
}
```

Exception: 0
Hello World
~B
~A



คลาส exception (C++ 1998/2003)

```
class exception {
public:
    exception () throw();
    exception (const exception&) throw();
    exception& operator= (const exception&) throw();
    virtual ~exception() throw();
    virtual const char* what() const throw();
};
```

คลาส exception (C++ 2011)

```
class exception {
public:
    exception () noexcept;
    exception (const exception&) noexcept;
    exception& operator= (const exception&) noexcept;
    virtual ~exception();
    virtual const char* what() const noexcept;
};
```

noexcept แตกต่างจาก **throw()** ตรงที่

- noexcept จะไม่เรียกใช้ **std::unexpected**
- และอาจจะยังไม่ได้คอมไพล์หรือไม่ก็ได้

เป็นการลดภาระที่เกิดจาก **throw()** ทำให้มีประสิทธิภาพมากขึ้น

โปรแกรมที่ 12.8

```
#include <iostream>
#include <exception>
using namespace std;

int main() {
    try {
        int *a = new int[1000000000];
    }
    catch (exception& e) {
        cout<<"Exception: "<<e.what()<<endl;
    }
    cout<<"Hello World"<<endl;
}
```

ผลลัพธ์โปรแกรม
Exception: StBad_alloc
Hello World

โปรแกรมที่ 12.9

```
#include <iostream>
#include <exception>
using namespace std;
class MyException: public exception {
public:
    virtual const char* what() const throw() {
        return "Hello my exception";
    }
};

void main() {
    MyException ex;
    try {
        cout<<"My world"<<endl;
        throw ex;
    }
    catch (exception& e) {
        cout << e.what() << endl;
    }
    cout<<"Hello World"<<endl;
}
```

ผลลัพธ์โปรแกรม
My world
Hello my exception
Hello world